

Deep Learning models are broadly classified into supervised and unsupervised models.

Supervised DL models:

- Artificial Neural Networks (ANNs)
- Recurrent Neural Networks (RNNs)
- Convolutional Neural Networks (CNNs)

Unsupervised DL models:

- Self Organizing Maps (SOMs)
- Boltzmann Machines
- Autoencoders

Boltzmann machine

- Boltzmann machine refers to an association of uniformly associated neuron-like structure that make hypothetical decisions about whether to be **on** or **off**.
- Boltzmann Machine was invented by renowned scientist **Geoffrey Hinton** and **Terry Sejnowski** in 1985.
- Boltzmann Machines have a fundamental learning algorithm that permits them to find exciting features that represent complex regularities in the training data.
- The learning algorithm is usually slow in networks with various layers of feature detectors, but it is quick in "**Restricted Boltzmann Machines**" that has a single layer of feature detectors.
- Many hidden layers can be adapted efficiently by comprising Boltzmann Machines, utilizing the feature activations of one as the training data for the next.

Boltzmann machine



Types of Boltzmann machines



**Restricted
Boltzmann Machines
(RBMs)**



**Deep Belief
Networks (DBNs)**

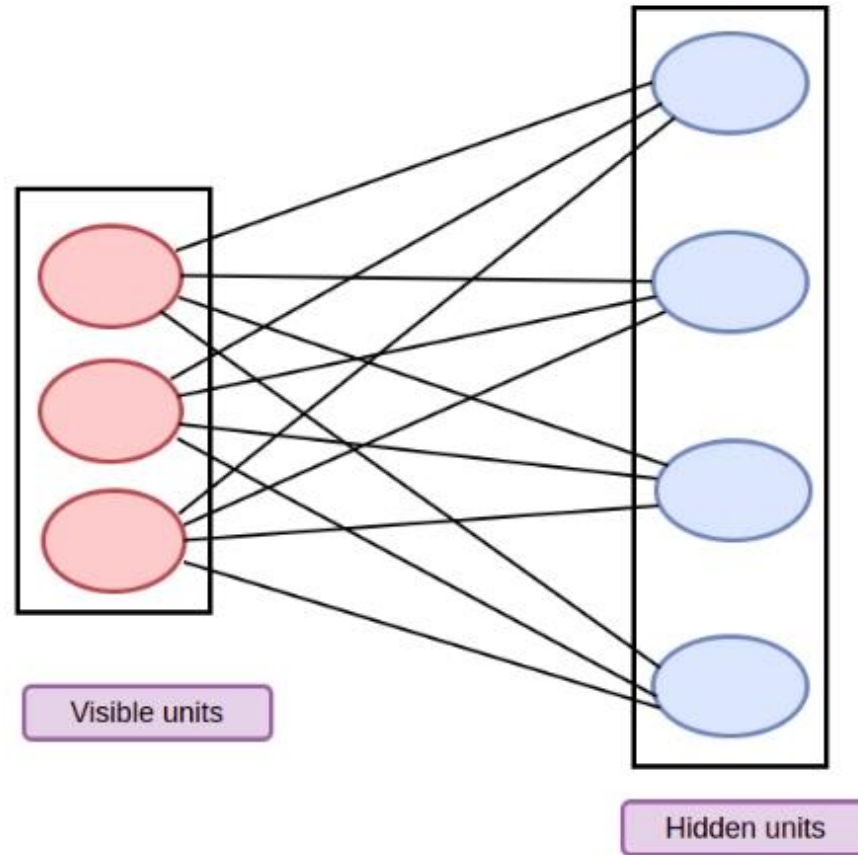


**Deep Boltzmann
Machines (DBMs)**

Restricted Boltzmann machine

- A restricted term means we cannot connect the same type of layer.
- In other words, the two neurons in the input or hidden layer cannot communicate even though the hidden and visible layers can be linked.
- Because there is no output layer in this machine, the question of how we will identify, update the weights, and measure whether our prediction is correct or not arises.
- All of the questions have a single answer: Restricted Boltzmann Machine.
- Geoffrey Hinton (2007) proposed the RBM algorithm, which learns probability distributions from sample training data inputs.
- It has widespread use in supervised and unsupervised machine learning applications such as feature learning, dimensionality reduction, classification, collaborative filtering, and topic modeling

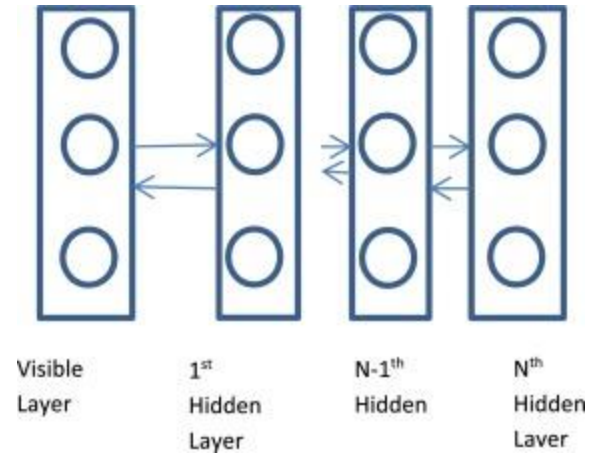
Restricted Boltzmann machine



Deep Boltzmann machine

- Deep Boltzmann machine is a model with additional hidden layers and directionless connections between the nodes.
- DBM learns features from raw data in a hierarchical manner, and the features recovered in one layer are applied as hidden variables as input to the subsequent layer.
- To define the training information, weight initialization, and adjustment parameters, the DBM training method must be modified.
- According to the DBM, temporal complexity limitations will occur when the parameters are set to ideal.
- Montavon et al presented a centering optimization strategy to make the learning process more robust and for mid-sized DBM to construct a generative, quicker, and discriminative model.

Deep Boltzmann machine

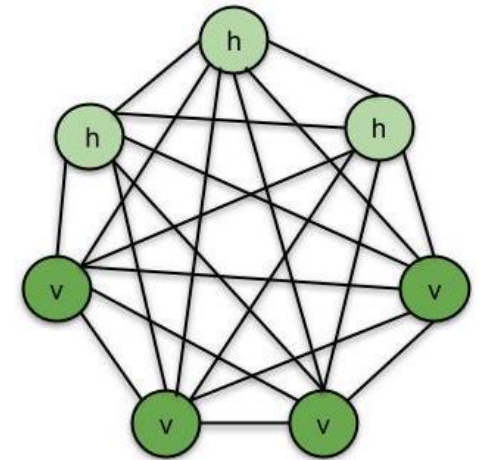


Deep Belief Network

- The Deep Belief Network is one of the types of Boltzmann machine.
- It is a generative model which uses multiple stacks of the deep architecture of the Restricted Boltzmann Machine.
- Each restricted Boltzmann machine performs a non-linear transformation on the input neurons and produces the outputs that serve as the input for the consecutive model.
- Deep Belief Networks can act supervised or unsupervised as they have a generative model. Due to it, Deep Belief Networks has a lot of flexibility, and it is easier to expand.

Deep Boltzmann machine

- A Boltzmann machine is an unsupervised deep learning model in which every node is connected to every other node. It is a type of recurrent neural network, and the nodes make binary decisions with some level of bias.
- These machines are not deterministic deep learning models, they are stochastic or [generative deep learning](#) models. They are representations of a system.
- A Boltzmann machine has two kinds of nodes
- Visible nodes:
These are nodes that can be measured and are measured.
- Hidden nodes:
These are nodes that cannot be measured and are not measured.



v - visible nodes, h - hidden nodes

Deep Boltzmann machine

- Boltzmann Machines are utilized to resolve two different computational issues.
- First, **for a search problem**, the weight on the associations are fixed and are used to represent a cost function.
- The stochastic dynamics of a Boltzmann Machine permit it to sample binary state vectors that have minimum values of the cost function.
- **Second, for a learning issue**, the Boltzmann Machine has indicated a set of binary data vectors, and this must figure out how to generate these vectors with high probability.
- To solve this, it must discover weights on the associations so that relative to other possible binary vectors, the data vectors have minimum values of the cost function.
- For solving a learning issue, Boltzmann machines make numerous small updates to their weights, and each update expects them to tackle a wide range of search issues.

Deep Belief Networks

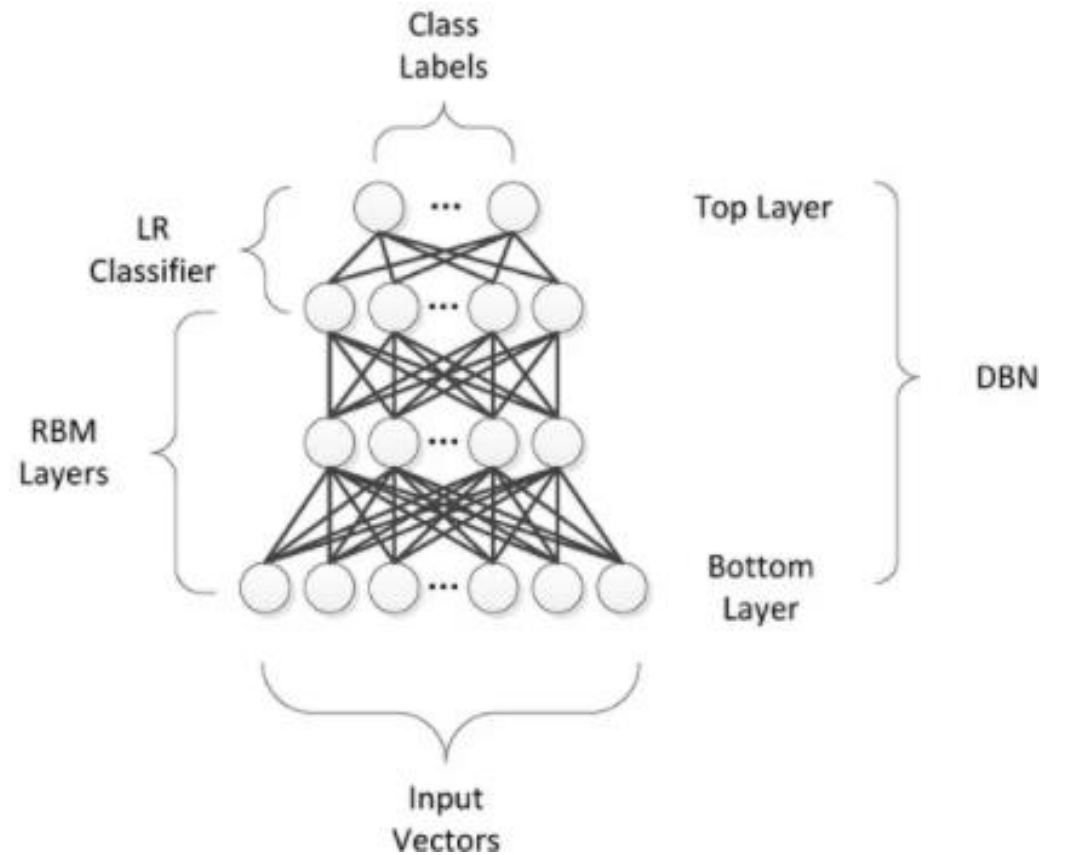
- Deep Belief Networks (DBNs) are a type of deep learning architecture combining unsupervised learning principles and neural networks.
- They are composed of layers of Restricted Boltzmann Machines (RBMs), which are trained one at a time in an unsupervised manner.
- The output of one RBM is used as the input to the next RBM, and the final output is used for supervised learning tasks such as classification or regression.

Deep Belief Networks

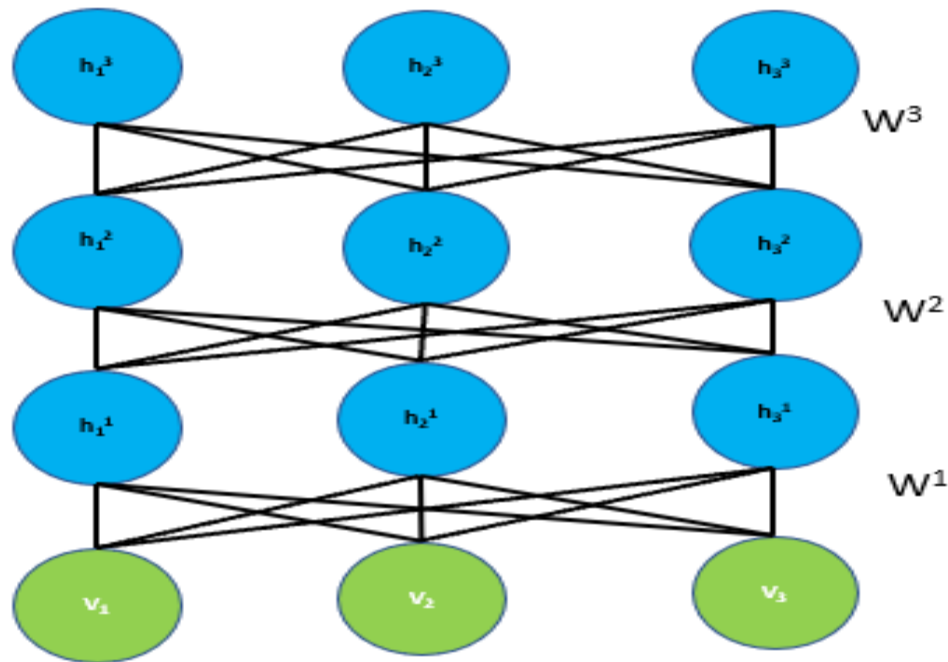
- *Deep Belief Networks (DBNs)* to address issues with classic neural networks in deep layered networks.
- For example – slow learning, becoming stuck in local minima owing to poor parameter selection, and requiring a large number of training datasets.

Deep Belief Networks

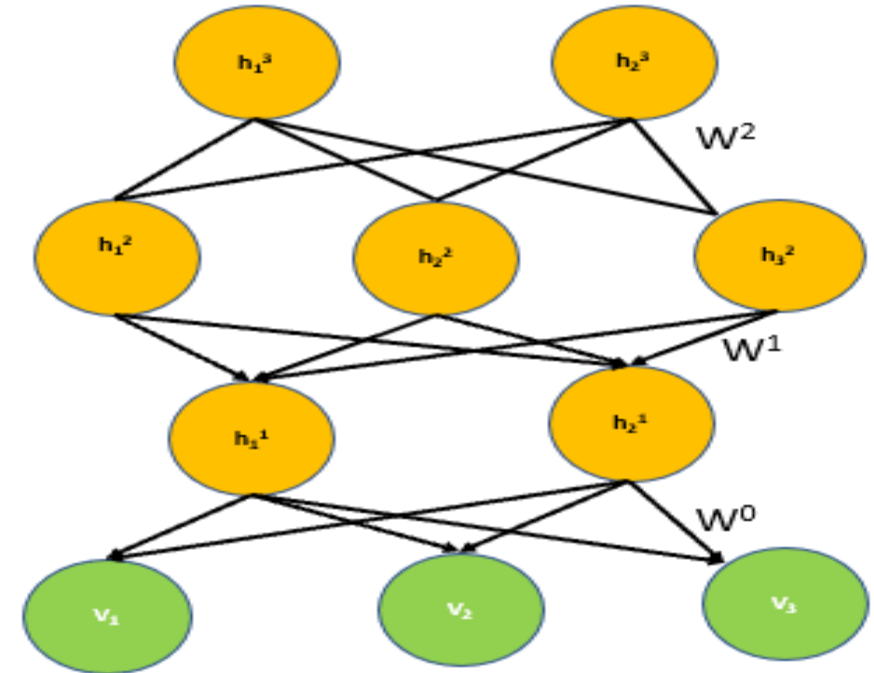
- Several layers of stochastic latent variables make a DBN.
- Binary latent variables that are often known as feature detectors or hidden units are binary variables.
- DBN is a **hybrid generative graphical** model.
- The **top two layers** have no direction.
- The layers above have directed links to **lower layers**.
- DBN is an algorithm for **unsupervised** probabilistic deep learning.



The Architecture of DBN



Deep Boltzmann Machine



Deep Belief Network

Deep Belief Networks

- A series of constrained Boltzmann machines connected in a specific order make a **Deep Belief Network**.
- Supplement the result of the “output” layer of the Boltzmann machine as input to the next Boltzmann machine consecutively.
- Then we’ll train it until its convergence and apply the same until the completion of the whole network.

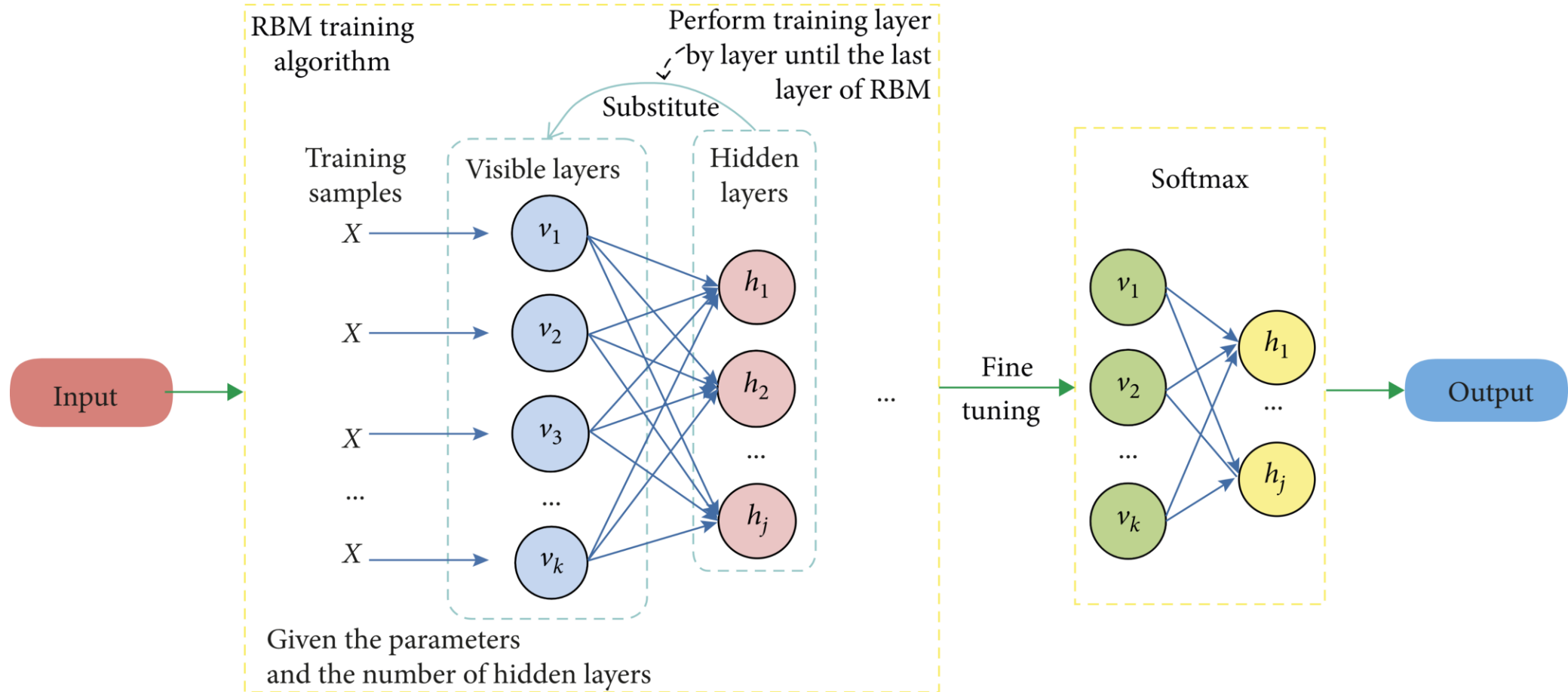
Deep Belief Networks

- The undirected and symmetric connections between the top two levels of DBN form associative memory.
- The arrows pointing towards the layer closest to the data point to the relationships between all lower layers.
- Directed acyclic connections in the lower layers translate associative memory to observable variables.
- The lowest layer of visible units receives the input data. We can use Binary or actual data as input.
- Like *Restricted Boltzmann Machine (RBM)*, there are no intralayer connections.
- The hidden units represent features that encapsulate the data's correlations.
- A matrix of symmetrical **weights $\mathbf{W}$** connects two layers.
- We'll link every unit in each layer to every other unit in the layer above it.

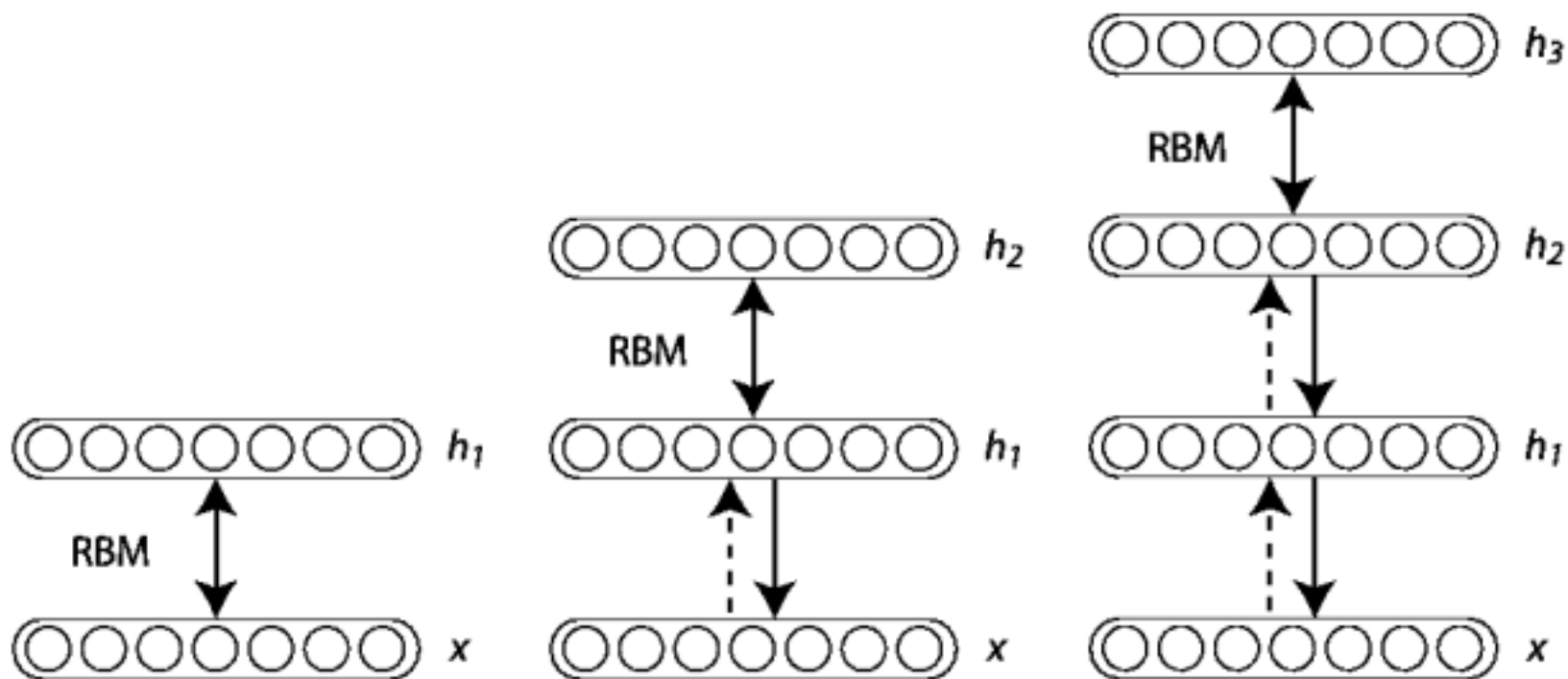
How does DBN work?

- The first stage is to train a property layer that can directly gain input signals from pixels.
- In an alternate retired subcaste, learn the features of the preliminarily attained features by treating the values of this subcaste as pixels.
- The lower bound on the log-liability of the training data set improves every time a fresh subcaste of parcels or features that we add to the network.

RBM WORKS



RBM



Deep Belief Network's operational pipeline is as follows:

- We'll use the Greedy learning algorithm to pre-train DBN. For learning the top-down generative weights-the greedy learning method that employs a layer-by-layer approach. These generative weights determine the relationship between variables in one layer and variables in the layer above.
- On the top two hidden layers, we run numerous steps of Gibbs sampling in DBN. The top two hidden layers define the RBM thus, this stage is effectively extracting a sample from it.
- Then generate a sample from the visible units using a single pass of ancestral sampling through the rest of the model.
- We'll use a single bottom-up pass to infer the values of the latent variables in each layer. In the bottom layer, greedy pretraining begins with an observed data vector. It then oppositely fine-tunes the generative weights.

Advantages and Disadvantages of RBM

- **Some of the Advantages of RBM Are:**
 - The hidden layer's activations can be included in other models as valuable features to boost performance.
 - Due to the limitations on connections between nodes, it is faster than a standard Boltzmann machine.
 - Efficiently computed and expressive enough to encode any distribution.
- **Some of the Disadvantages of RBM Are:**
 - The backpropagation algorithm is more well known than the CD-k algorithm, which is utilized in RBMs.
 - Because it is challenging to calculate the Energy gradient function, training is more challenging.
 - Weight Modification.

Applications of Restricted Boltzmann Machine

- **Collaborative filtering:** RBMs are widely used in collaborative filtering for recommender systems. They learn to predict user preferences based on their past behavior and recommend items that are likely to be of interest to the user.
- **Image and video processing:** RBMs can be used for image and video processing tasks such as object recognition, image denoising, and image reconstruction. They can also be used for tasks such as video segmentation and tracking.
- **Natural language processing:** RBMs can be used for natural language processing tasks such as language modeling, text classification, and sentiment analysis. They can also be used for tasks such as speech recognition and speech synthesis.

Applications of Restricted Boltzmann Machine

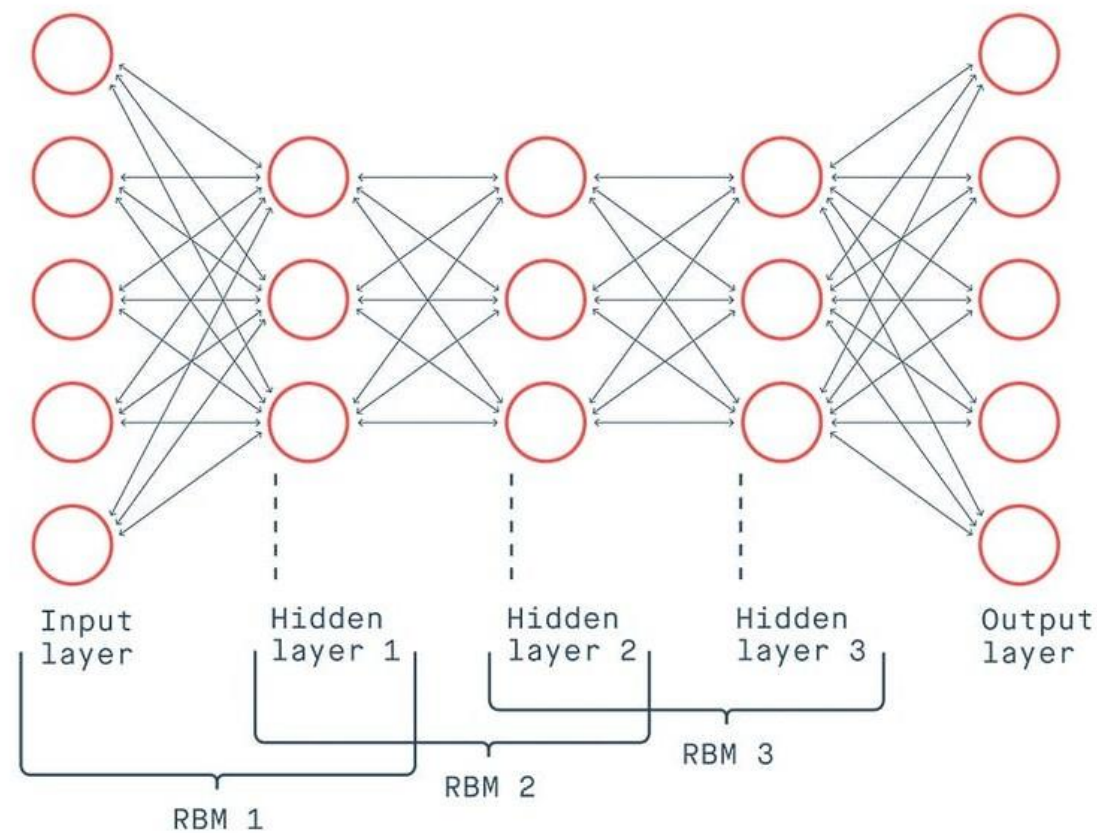
- **Bioinformatics:** RBMs have found applications in bioinformatics for tasks such as protein structure prediction, gene expression analysis, and drug discovery.
- **Financial modeling:** RBMs can be used for financial modeling tasks such as predicting stock prices, risk analysis, and portfolio optimization.
- **Anomaly detection:** RBMs can be used for anomaly detection tasks such as fraud detection in financial transactions, network intrusion detection, and medical diagnosis.
- It is used in Filtering.
- It is used in Feature Learning.
- It is used in Classification.
- It is used in Risk Detection.
- It is used in Business and Economic analysis.

- It's necessary to remember that constructing a Deep Belief Network necessitates training each RBM layer.
- Initially, we'll initiate the units and parameters for this purpose.
- In the Contrastive Divergence algorithm, there are two phases: **positive and negative**.
- We'll calculate the binary states of the hidden layers in the positive phase by computing the probabilities of weights and visible units.
- It is known as the positive phase since it enhances the likelihood of the training data set.
- The negative phase reduces the likelihood of the model producing samples.
- To train a complete Deep Belief Network, we'll employ the greedy learning technique. The greedy learning algorithm trains one RBM at a time until all of the RBMs are trained.

Creating a Deep Belief Network

- Several RBMs together make a Deep Belief Networks, which reflects in their Clojure record structure. We use a fully unsupervised form of DBN to initialize a Deep Neural Network, whereas we use a classification DBN (CDBN) as a classification model on its own.

Creating a Deep Belief Network



Deep Belief Network

- Each record type includes the RBMs that make up the network's layers, as well as a vector- indicating the layer size and, in the case of the classification DBN- the number of classes in the representative dataset.
- The DBN record represents a model made up entirely of stacked RBMs, but the CDBN record contains the top-level associative memory, which is a CRBM record.
- This enables the top layer to be trained to create class labels for input data vectors and to classify unknown data vectors.

Deep Belief Network

- Deep Belief Networks (DBNs) are a type of artificial neural network that are used for unsupervised and [supervised learning tasks](#).
- They are composed of several layers of [Restricted Boltzmann Machines \(RBMs\)](#), which are shallow neural networks that can be trained using [unsupervised learning](#).
- The output of the RBMs is then used as input to the next layer of the network, until the final layer is reached.
- The final layer of the DBN is typically a classifier that is trained using supervised learning.

Applications of Deep Belief Networks

- 1.Computer vision:** DBNs have been used for tasks like recognizing objects in pictures or putting pictures into different groups.
- 2.Speech recognition:** DBNs have been used for speech recognition tasks, such as transcribing speech into text.
- 3.Natural language processing:** DBNs have been used for natural languages processing tasks, such as sentiment analysis and text classification.
- 4.Recommender systems:** DBNs have been used in recommender systems, which give users suggestions based on what they like and how they act.
- 5.In bioinformatics,** DBNs have been used to predict how proteins will interact with each other, look at how genes are expressed, and find new drugs.
- 6.Financial analysis:** DBNs have been used to predict the stock market and figure out how risky something is.

Benefits of Deep Belief Networks

- DBN is efficient in the usage of hidden layers (higher performance gain by adding layers compared to Multilayer perceptron).
- DBN has specific robustness in classification (size, position, color, view angle – rotation).
- DBN's same neural network approach can be implemented on various applications and data types.
- Even when the amount of data is huge, DBN provides the best performance results.
- DBN's features are habitually gathered & optimally tuned for anticipated outcomes.
- DBN avoids time taking techniques.
- Last but not least, DBN is flexible to new problems in the future.

Drawbacks of Deep Belief Networks

- 1.DBN has hardware requirements.
- 2.DBN requires huge data to perform better techniques.
- 3.DBN is expensive to train because it has complex data models.
- 4.Hundreds of machines are required.
- 5.DBN is difficult to be used by less skilled people.
- 6.DBN requires classifiers to grasp the output.

Overview

- A DBN is sometimes narrated as a stack of Restricted Boltzmann machines (RBMs) placed on top of one another.
- We create Deep Belief Networks (DBNs) to address issues with classic neural networks in deep layered networks.
- A number of smaller unsupervised neural networks makes up a Deep belief networks.
- We can calculate the binary states of the hidden layers in the positive phase by computing the probabilities of weights and visible units.
- Although the layers are connected the network does not have connections between units inside a single layer, which is a common feature of deep belief networks.
- It's necessary to remember that constructing a Deep Belief Network necessitates training each RBM layer.
- The greedy learning algorithm trains one RBM at a time until all of the RBMs are trained.

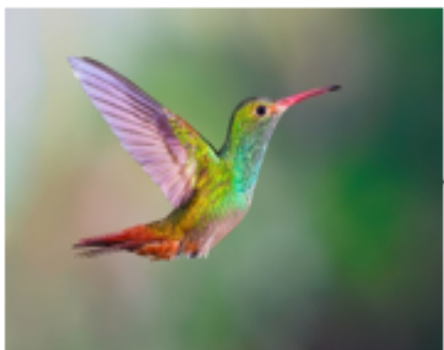
UNIT 4

OPTIMIZATION ON DEEP MODELS: Optimization for Training Deep Models: Challenges in Neural Networks optimization - Basic Algorithms - Algorithms Adaptive learning Rates - Approximate Second Order Methods - Optimization Strategies and Meta Algorithms - Convolutional Networks Motivation - Structured Output - Unsupervised features - Neuroscientific basics for Convolutional Networks.

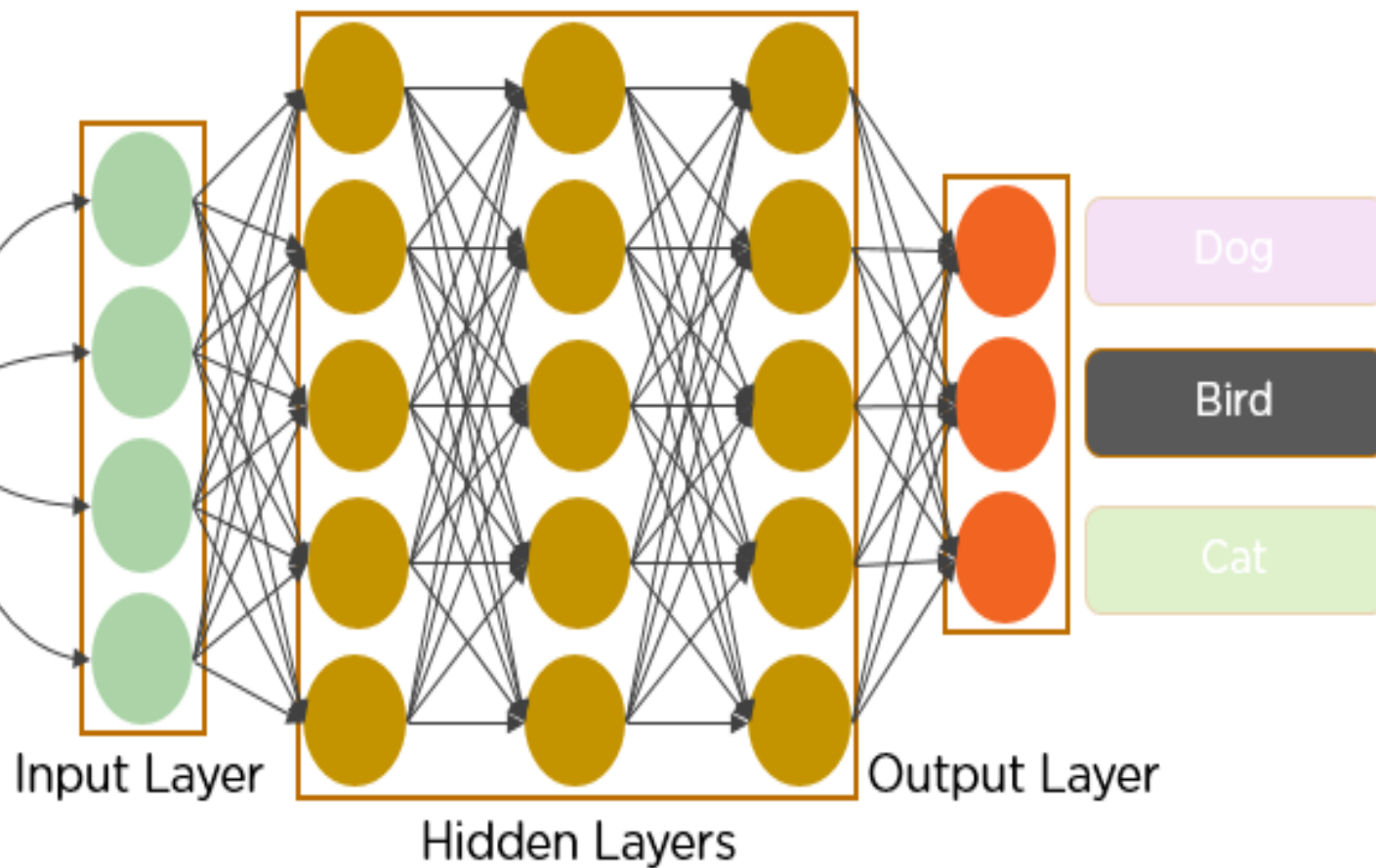
Convolutional Networks

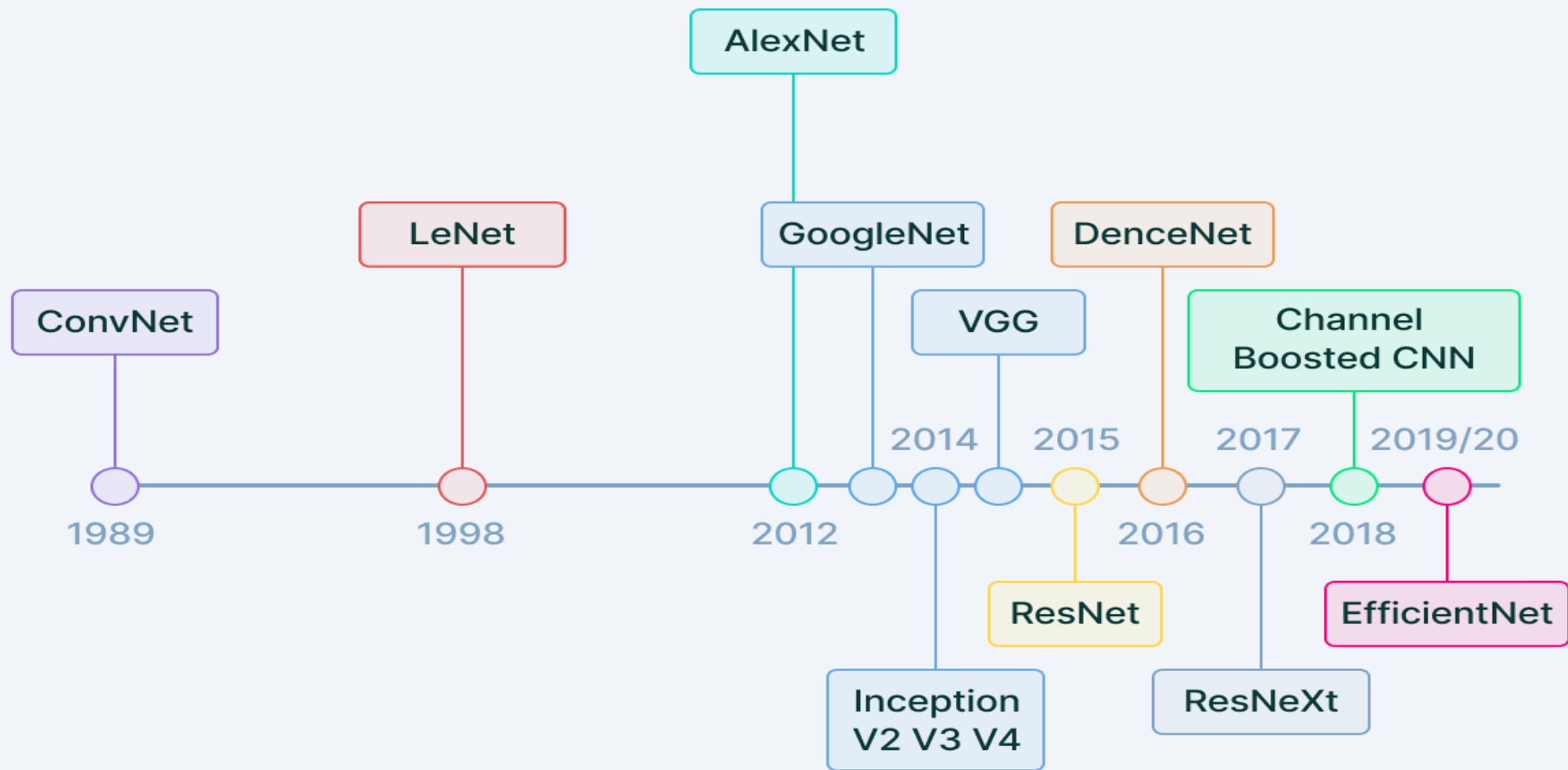
- Deep Learning has proved to be a very powerful tool because of its ability to handle large amounts of data.
- The interest to use hidden layers has surpassed traditional techniques, especially in pattern recognition.
- One of the most popular deep neural networks is Convolutional Neural Networks (also known as CNN or ConvNet) in deep learning, especially when it comes to Computer Vision applications.

- **Convolutional networks**, also known as **convolutional neural networks** or CNNs, are a specialized kind of neural network for processing data that has a known, grid-like topology.
- Examples include time-series data, which can be thought of as a 1D grid taking samples at regular time intervals, and image data, which can be thought of as a 2D grid of pixels.
- Convolutional networks have been tremendously successful in practical applications.
- The name “convolutional neural network” indicates that the network employs a mathematical operation called **convolution**.
- Convolution is a specialized kind of linear operation. *Convolutional networks are simply neural networks that use convolution in place of general matrix multiplication in at least one of their layers.*



Pixels of image fed as input





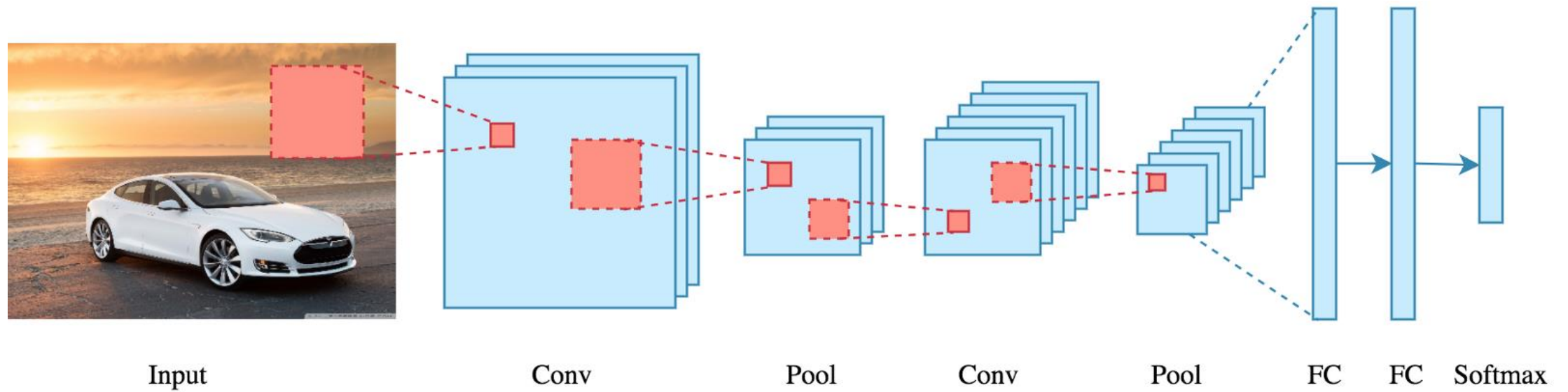
Background of CNNs

- CNN's were first developed and used around the 1980s.
- The most that a CNN could do at that time was recognize handwritten digits.
- It was mostly used in the postal sectors to read zip codes, pin codes, etc.
- The important thing to remember about any deep learning model is that it requires a large amount of data to train and also requires a lot of computing resources.
- This was a major drawback for CNNs at that period and hence CNNs were only limited to the postal sectors and it failed to enter the world of machine learning.

- In 2012 Alex Krizhevsky realized that it was time to bring back the branch of deep learning that uses multi-layered neural networks.
- The availability of large sets of data, to be more specific ImageNet datasets with millions of labeled images and an abundance of computing resources enabled researchers to revive CNNs.

What Is a CNN?

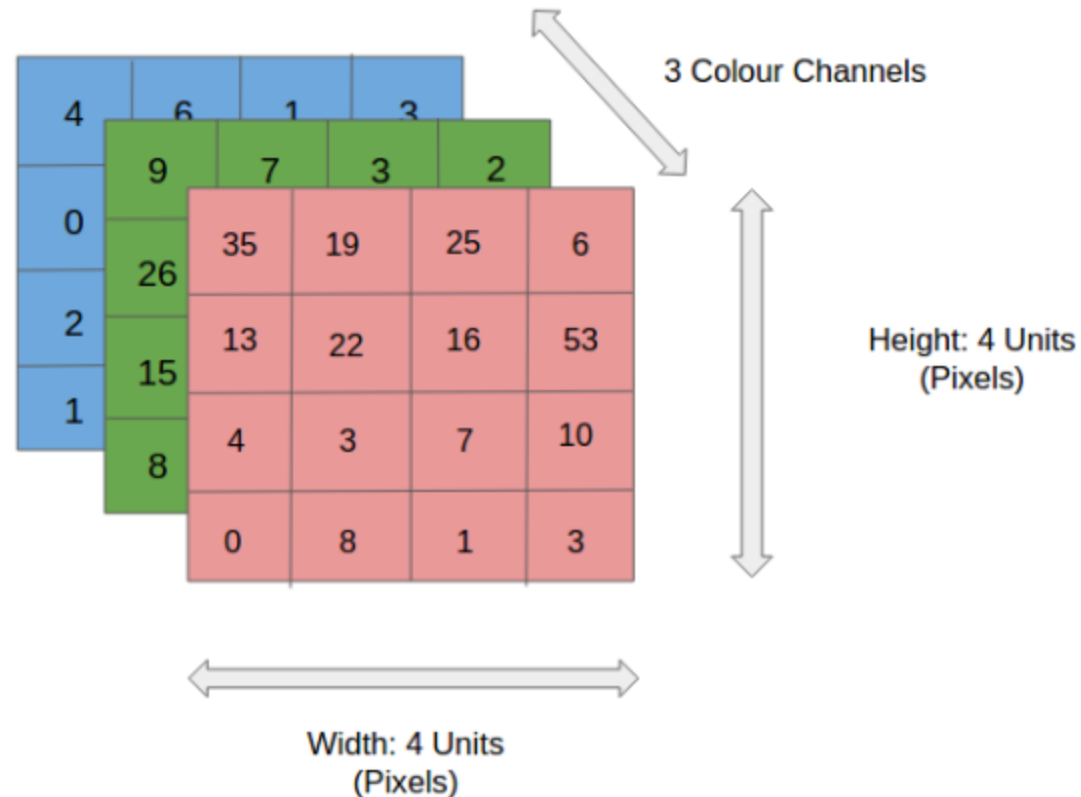
- In deep learning, a **convolutional neural network (CNN/ConvNet)** is a class of [deep neural networks](#), most commonly applied to analyze visual imagery.
- Now when we think of a neural network we think about matrix multiplications but that is not the case with ConvNet.
- It uses a special technique called Convolution.
- Now in mathematics **convolution** is a mathematical operation on two functions that produces a third function that expresses how the shape of one is modified by the other.

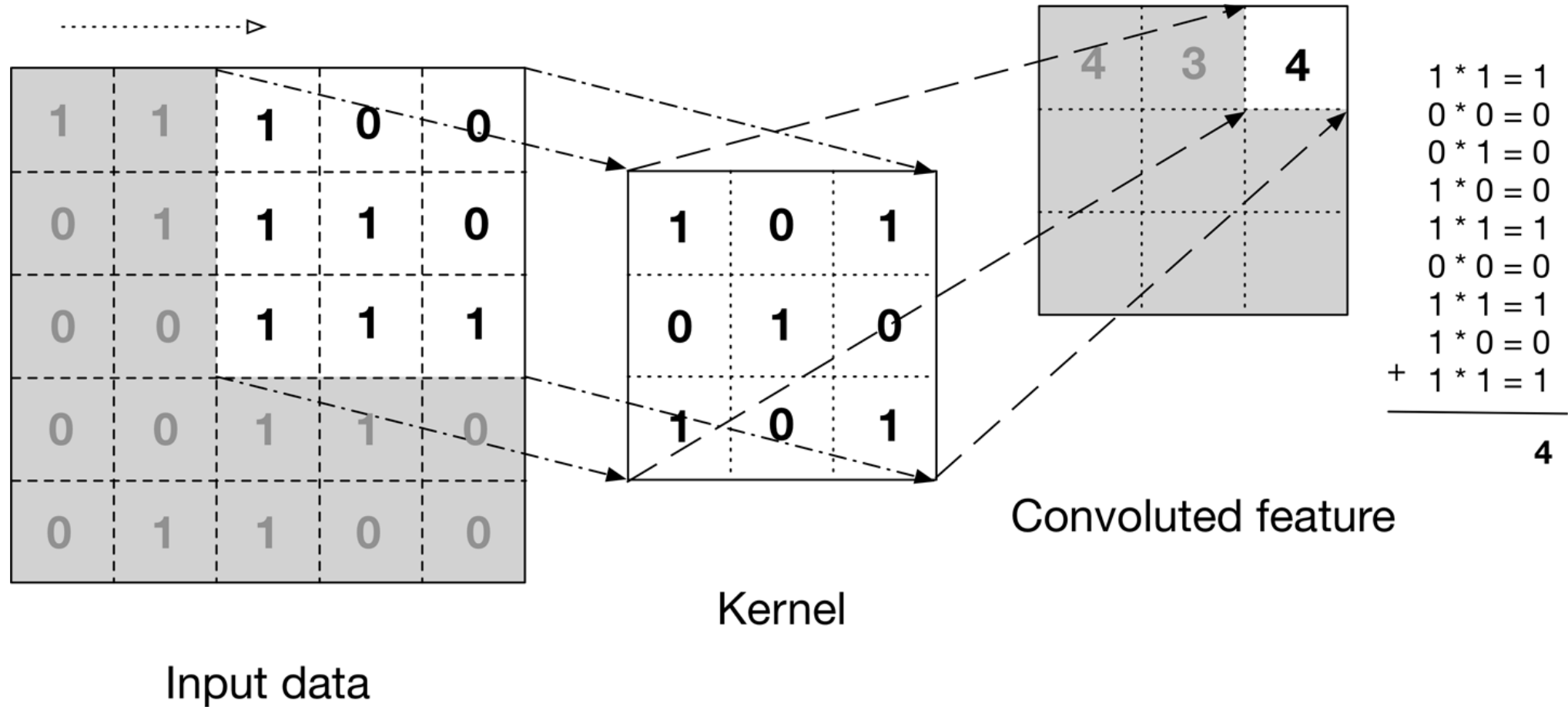


But we don't really need to go behind the mathematics part to understand what a CNN is or how it works.
Bottom line is that the role of the ConvNet is to reduce the images into a form that is easier to process, without losing features that are critical for getting a good prediction.

How does it work?

- An RGB image is nothing but a matrix of pixel values having three planes whereas a grayscale image is the same but it has a single plane.





1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1



308

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2



-498

0	1	1
0	1	0
1	-1	1

Kernel Channel #3



164

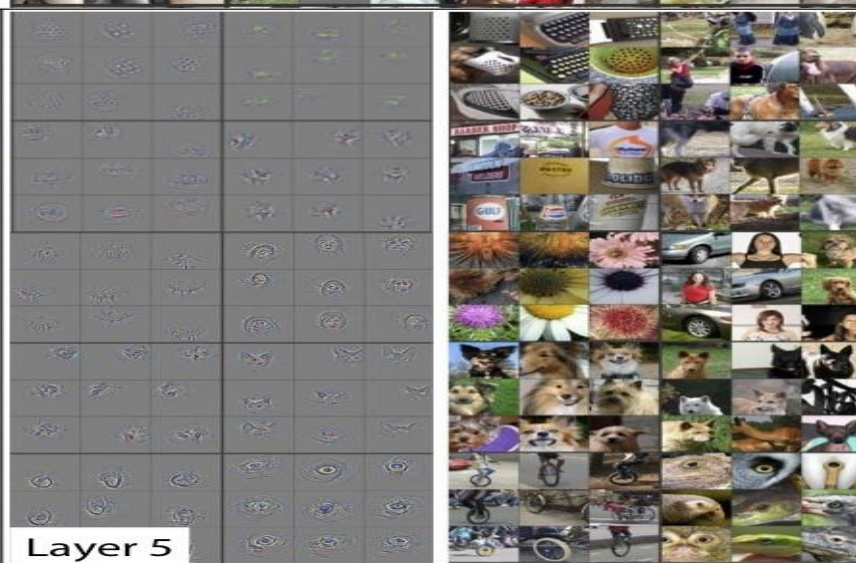
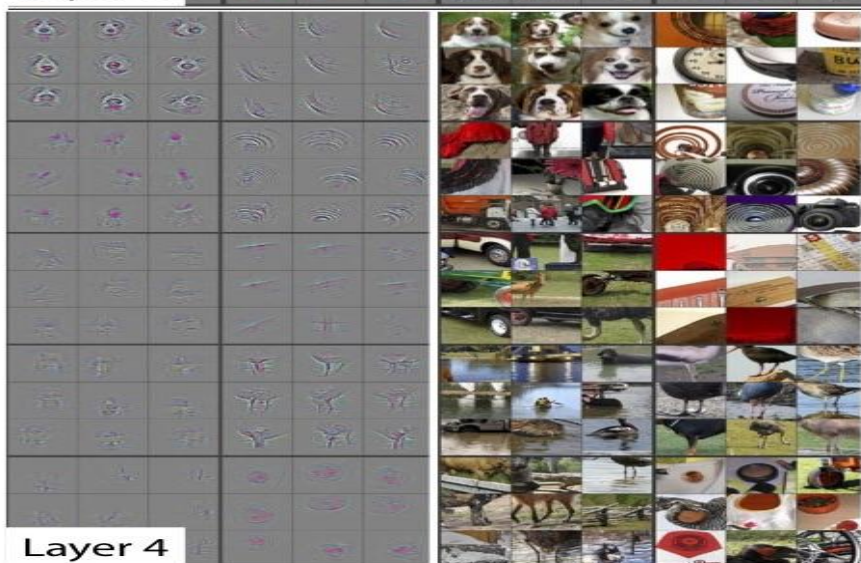
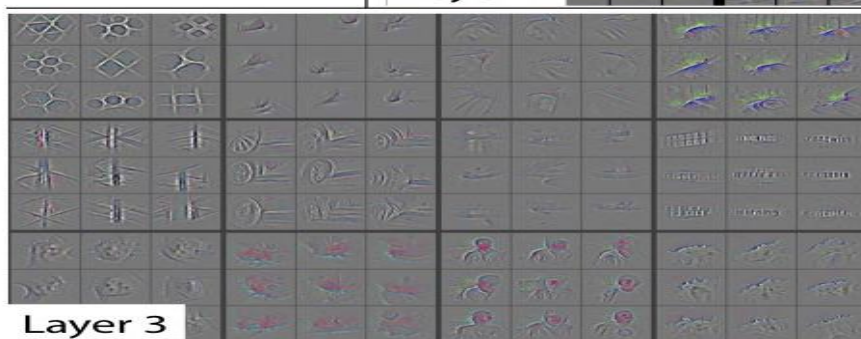
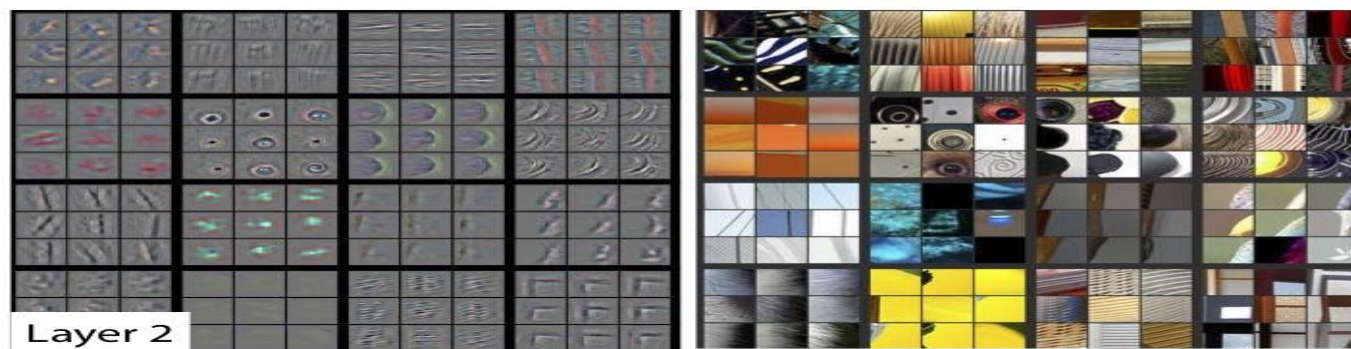
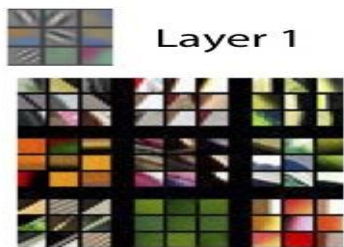
+ 1 = -25

Bias = 1

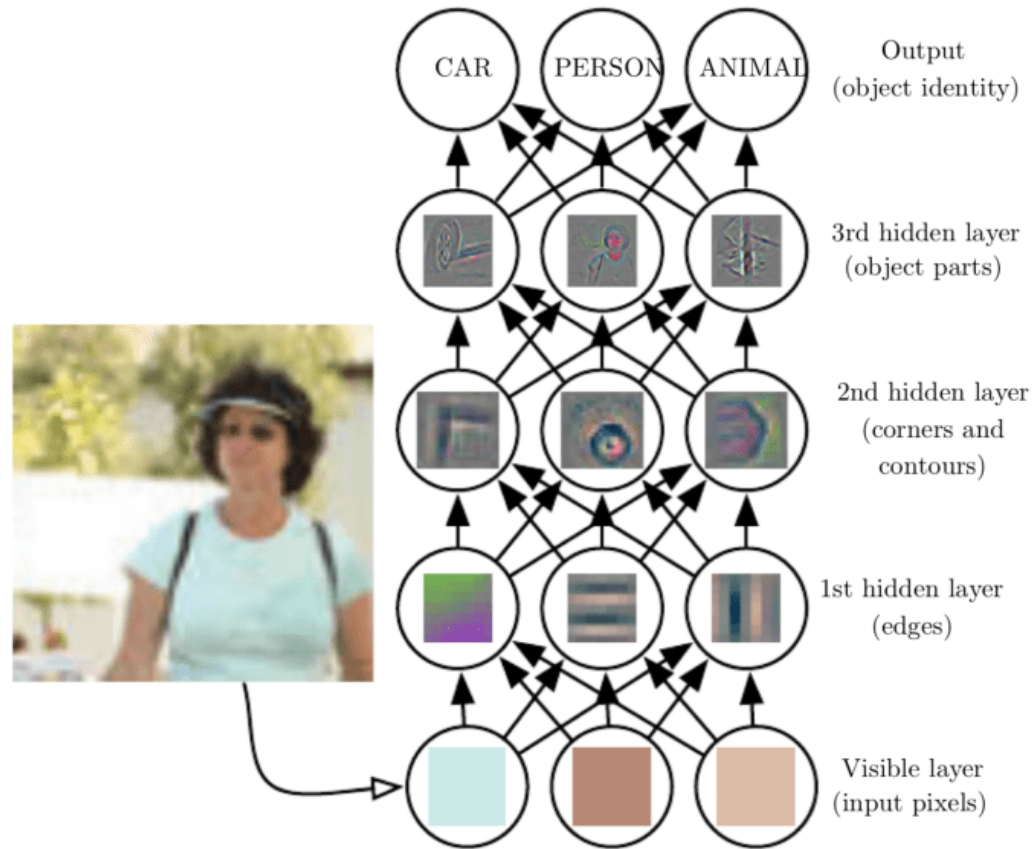
Output

-25				...
				...
				...
				...
...	...	...	...	...

- Convolutional neural networks are composed of multiple layers of artificial neurons.
- Artificial neurons, a rough imitation of their biological counterparts, are mathematical functions that calculate the weighted sum of multiple inputs and outputs an activation value.
- When you input an image in a ConvNet, each layer generates several activation functions that are passed on to the next layer.
- The first layer usually extracts basic features such as horizontal or diagonal edges.
- This output is passed on to the next layer which detects more complex features such as corners or combinational edges. As we move deeper into the network it can identify even more complex features such as objects, faces, etc.



- Based on the activation map of the final convolution layer, the classification layer outputs a set of confidence scores (values between 0 and 1) that specify how likely the image is to belong to a “class.”
- For instance, if you have a ConvNet that detects cats, dogs, and horses, the output of the final layer is the possibility that the input image contains any of those animals.



What Is a Pooling Layer?

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

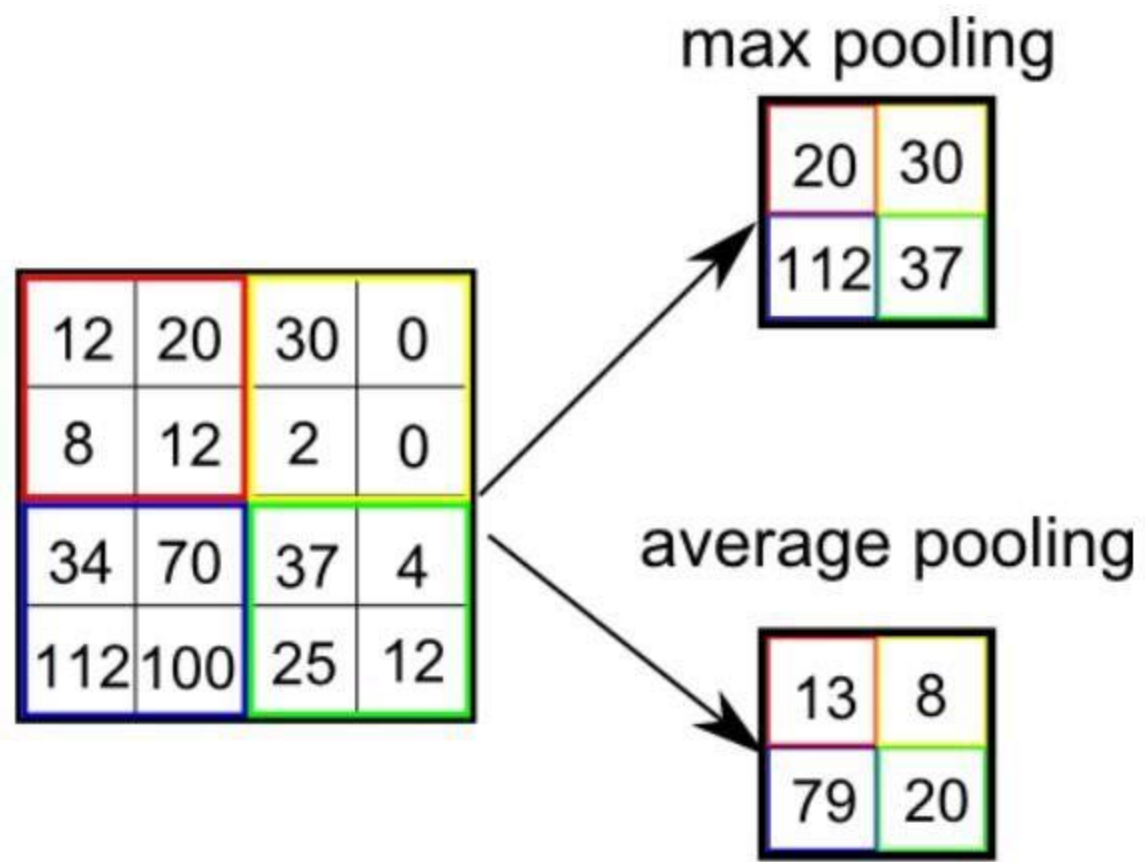
3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

- The purpose of the pooling layers is to reduce the dimensions of the hidden layer by combining the outputs of neuron clusters at the previous layer into a single neuron in the next layer.

Why to use Pooling Layers?

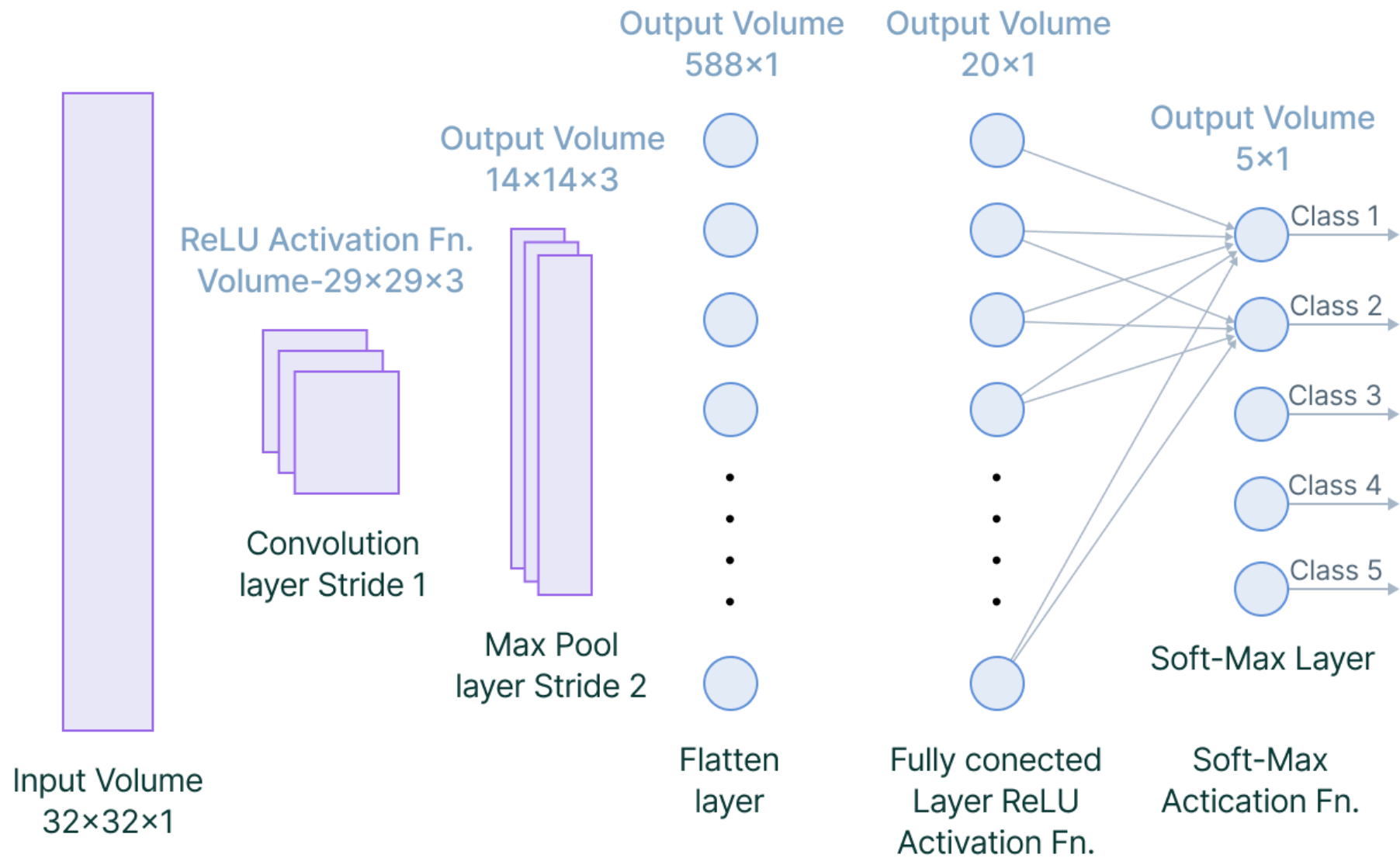
- Pooling layers are used to reduce the dimensions of the feature maps.
- Thus, it reduces the number of parameters to learn and the amount of computation performed in the network.
- The pooling layer summarises the features present in a region of the feature map generated by a convolution layer.
- So, further operations are performed on summarised features instead of precisely positioned features generated by the convolution layer.
- This makes the model more robust to variations in the position of the features in the input image

- Max Pooling is we find the maximum value of a pixel from a portion of the image covered by the kernel. Max Pooling also performs as a **Noise Suppressant**. It discards the noisy activations altogether and also performs de-noising along with dimensionality reduction.
- On the other hand, **Average Pooling** returns the **average of all the values** from the portion of the image covered by the Kernel. Average Pooling simply performs dimensionality reduction as a noise suppressing mechanism. Hence, we can say that **Max Pooling performs a lot better than Average Pooling**.



Fully-Connected Layer

- The Convolutional Layer, along with the Pooling Layer, forms a block in the Convolutional Neural Network.
- The number of such layers may be increased for capturing finer details depending upon the complexity of the task at the cost of more computational power.



Advantages & Disadvantages of CNNs

- **Advantages of Convolutional Neural Networks (CNNs):**

1. Good at detecting patterns and features in images, videos, and audio signals.
2. Robust to translation, rotation, and scaling invariance.
3. End-to-end training, no need for manual feature extraction.
4. Can handle large amounts of data and achieve high accuracy.

- **Disadvantages of Convolutional Neural Networks (CNNs):**

1. Computationally expensive to train and require a lot of memory.
2. Can be prone to overfitting if not enough data or proper regularization is used.
3. Requires large amounts of labeled data.
4. Interpretability is limited, it's hard to understand what the network has learned.

Convolutional Neural Networks: Summary

- The Convolutional Neural Network is a type of artificial neural network commonly applied in image processing problems.
- A fully connected neural network involves far more computations than a ConvNet ,which does not work well in images.
- There are a couple of layers that make CNN unique—the convolutional layer and the Pooling layer.
- The convolutional layer works by placing a filter over an array of image pixels and creates a convolved feature map. It is simply looking at an image through a window that allows you to see the presence of specific features.
- Pooling layer down samples and reduces the size of the feature map.
- There are two types of pooling- Max Pooling that takes the maximum value from a particular convolved input or Average Pooling that simply takes the average of all the values.
- A fully connected layer is needed, which takes input from the flattened feature map to perform classification.
- A Convolutional Neural Network is a feedforward network that filters spatial data.

The Neuroscientific Basis for Convolutional Networks

- Neurophysiologists David Hubel and Torsten Wiesel collaborated for several years to determine many of the most basic facts about how the mammalian vision system works (Hubel and Wiesel, 1959, 1962, 1968).
- Their accomplishments were eventually recognized with a Nobel prize.
- Their findings that have had the greatest influence on contemporary deep learning models were based on recording the activity of individual neurons in cats.
- They observed how neurons in the cat's brain responded to images projected in precise locations on a screen in front of the cat.
- Their great discovery was that neurons in the early visual system responded most strongly to very specific patterns of light, such as precisely oriented bars, but responded hardly at all to other patterns.

- V1, also known as the **primary visual cortex**.
- V1 is the first area of the brain that begins to perform significantly advanced processing of visual input.
- In this cartoon view, images are formed by light arriving in the eye and stimulating the retina, the light-sensitive tissue in the back of the eye.
- The neurons in the retina perform some simple preprocessing of the image but do not substantially alter the way it is represented.
- **The image then passes through the optic nerve and a brain region called the lateral geniculate nucleus.**
- The main role, as far as we are concerned here, of both of these anatomical regions is primarily just to carry the signal from the eye to V1, which is located at the back of the head.

A convolutional network layer is designed to capture three properties of V1

- V1 is arranged in a **spatial map**.
- It actually has a two-dimensional structure mirroring the structure of the image in the retina.
- For example, light arriving at the lower half of the retina affects only the corresponding half of V1.
- Convolutional networks capture this property by having their features defined in terms of two dimensional maps.

A convolutional network layer is designed to capture three properties of V1

- V1 contains many **simple cells**.
- A simple cell's activity can to some extent be characterized by a linear function of the image in a small, spatially localized receptive field.
- The detector units of a convolutional network are designed to emulate these properties of simple cells.

A convolutional network layer is designed to capture three properties of V1

- V1 also contains many **complex cells**.
- These cells respond to features that are similar to those detected by simple cells, but complex cells are invariant to small shifts in the position of the feature.
- This inspires the pooling units of convolutional networks.
- Complex cells are also invariant to some changes in lighting that cannot be captured simply by pooling over spatial locations.
- These invariances have inspired some of the cross-channel pooling strategies in convolutional networks, such as maxout units ([Goodfellow et al., 2013a](#)).

Common Challenges in Deep Learning Optimization

- **Local Minima:** The grandfather of all optimization problems, local minima is a permanent challenge in the optimization of any deep learning algorithm. The local minima problem arises when the gradient encounters many local minimums that are different and not correlated to a global minimum for the cost function.
- **Flat Regions:** In deep learning optimization models, flat regions are common areas that represent both a local minimum for a sub-region and a local maximum for another. That duality often causes the gradient to get stuck.
- **Inexact Gradients:** There are many deep learning models in which the cost function is intractable which forces an inexact estimation of the gradient. In these cases, the inexact gradients introduces a second layer of uncertainty in the model.
- **Local vs. Global Structures:** Another very common challenge in the optimization of deep learning models is that local regions of the cost function don't correspond with its global structure producing a misleading gradient.

Challenges with Optimization

- [Convex optimization](#) involves a function in which there is only one optimum, corresponding to the global optimum (maximum or minimum).
- There is no concept of local optima for convex optimization problems, making them relatively easy to solve — these are common introductory topics in undergraduate and graduate optimization classes.
- *Non-convex optimization* involves a function which has multiple optima, only one of which is the global optima. Depending on the loss surface, it can be very difficult to locate the global optima.

There are multiple problems associated with this:

- **What is a reasonable learning rate to use?**
- Too small a learning rate takes too long to converge, and too large a learning rate will mean that the network will not converge.
- **How do we avoid getting stuck in local optima?**
- One local optimum may be surrounded by a particularly steep loss function, and it may be difficult to ‘escape’ this local optimum.

What if the loss surface morphology changes?

- Even if we can find the global minimum, there is no guarantee that it will remain the global minimum indefinitely.
- A good example of this is when training on a dataset that is not representative of the actual data distribution — when applied to new data, the loss surface will differ.
- This is one reason why trying to make the training and test datasets representative of the total data distribution is of such high importance.
- Another good example is data which habitually changes in distribution due to its dynamic nature — an example of this would be user preferences for popular music or movies, which changes day-to-day and month-to-month.

Analysis and explanation of deep learning-related technologies.

Main techniques of deep learning	Analysis and explanation of related contents
Stochastic forest model	The main branch of deep learning is also the most representative decision tree model
Support vector machine	A learning algorithm for complex data processing and regression
Multi perceptron	A typical feed forward artificial neural network model
Convolution neural network	A deep neural network framework with convolution structure
Activation function	A core hyper of nonlinear fitting can directly affect the data processing ability of the deep learning model for nonlinear mechanism
Loss function	A deep learning model fits the data to be the state representation of the fitted data
Model optimization	Training optimization of complex set of parameters and functions of the deep learning model

Related parameters of the deep learning network layer in multimedia data processing

Deep learning network layers	Operation type	Filter (step size)	Output size
In-4a	Maximum pooling of data	33/2	55515
In-4e	Maximum pooling of data	34/3	66512
In-SB	Maximum pooling of data	33/2	77512
Ip-4a	Data convolution	33/1	661024
Ip-4e	Data convolution	33/1	44256
Ip-5e	Data convolution	32/3	5525
Merging layer	Data tandem	44/1	44768
Convolution layer	Data convolution	112	66128
Pool layer	Data summation pooling	11/5	11128
Depth characteristic layer	Data characterization	150	11150

Related components of multimedia data processing technology

Case base names	Typical case	Emphasis
Multimedia data representation	Face image	Data acquisition display
Digital image processing	Face detection	Human-computer interaction data
Speech signal processing	Speech recognition in human-computer interaction	Education and training
Multimedia data compression	Compressed sensing technology and its application	Data storage
Multimedia data retrieval	Feature extraction of multimedia data	Multimedia data transmission retrieval
Multimedia information retrieval	Multimedia data matching	Multimedia image retrieval
Multimedia fusion data processing	Multi human-computer interaction multimedia data	Family entertainment

Correlation analysis and explanation of multimedia data characteristics

Multimedia data characteristics	Analysis and interpretation of multimedia related features
Quantitative	The amount of multimedia data are very large
Integration	Multimedia data are mainly composed of a variety of different types of media
Interactivity	Multimedia data can be processed and created
Real-time	Multimedia is a kind of time-based media with time characteristics
Nonstructural	Multimedia data have different coding methods and their respective processing methods and structures
Dynamics	Multimedia data have the characteristics of random combination/addition/deletion
Nonlinear	Multimedia data are presented in a flexible and varied manner
Control	Multimedia data can be controlled and expressed in the form of demand

Optimization

- In deep learning, we have the concept of loss, which tells us how poorly the model is performing at that current instant.
- Now we need to use this loss to **train** our network such that it performs better.
- Essentially what we need to do is to take the loss and try to **minimize** it, because a lower loss means our model is going to perform better.
- The process of minimizing (or maximizing) any mathematical expression is called **optimization**.

- Optimizers are algorithms or methods used to change the attributes of the neural network such as **weights** and **learning rate** to reduce the losses.
- Optimizers are used to solve optimization problems by minimizing the function.
- Optimization algorithms are responsible for reducing the losses and to provide the most accurate results possible.

Different types of optimizers and how they exactly work to minimize the loss function.

- 1.Gradient Descent
- 2.Stochastic Gradient Descent (SGD)
- 3.Mini Batch Stochastic Gradient Descent (MB-SGD)
- 4.SGD with momentum
- 5.Nesterov Accelerated Gradient (NAG)
- 6.Adaptive Gradient (AdaGrad)
- 7.AdaDelta
- 8.RMSprop
- 9.Adam

Gradient Descent

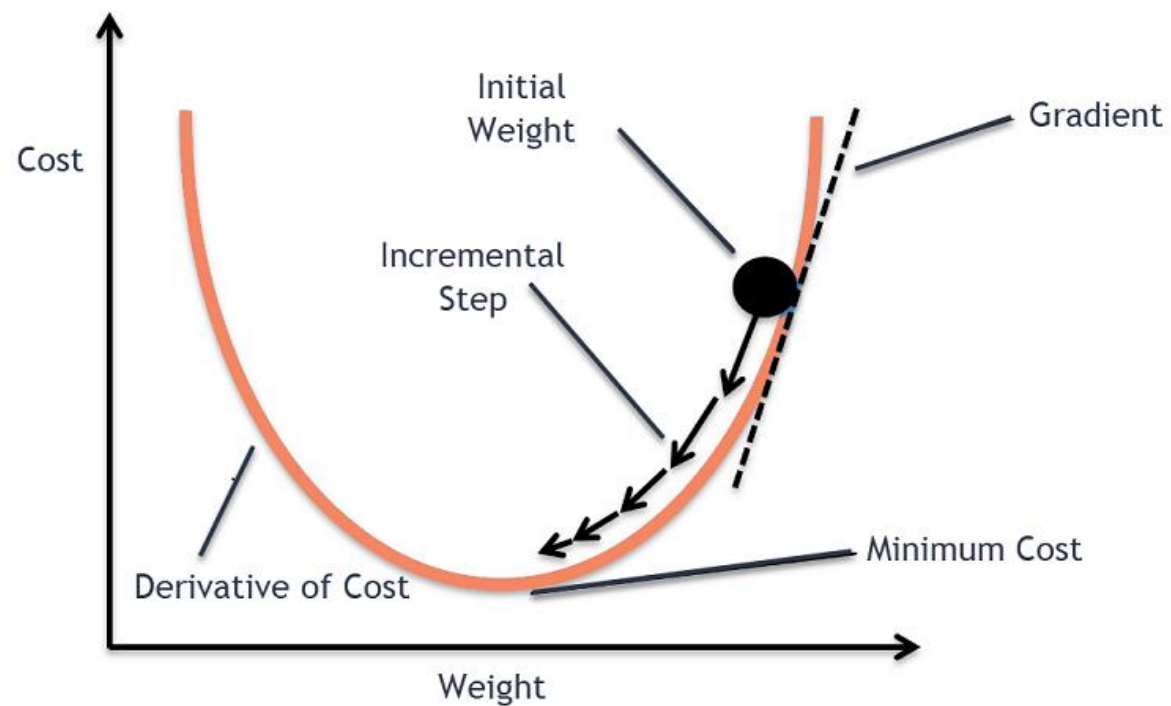
- Gradient descent is an optimization algorithm that's used when training a machine learning model.
- It's based on a convex function and tweaks its parameters iteratively to minimize a given function to its local minimum.

WHAT IS GRADIENT DESCENT?

- Gradient Descent is an optimization algorithm for finding a local minimum of a differentiable function.
- Gradient descent is simply used to find the values of a function's parameters (coefficients) that minimize a cost function as far as possible.
- <https://www.kdnuggets.com/2020/12/optimization-algorithms-neural-networks.html>

The goal of the gradient descent algorithm is to minimize the given function (say cost function). To achieve this goal, it performs two steps iteratively:

1. **Compute the gradient** (slope), the first order derivative of the function at that point
2. **Make a step (move) in the direction opposite to the gradient**, opposite direction of slope increase from the current point by alpha times the gradient at that point.
3. Alpha is called **Learning rate** — a tuning parameter in the optimization process. It decides the length of the steps.



Repeat until convergence {

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

}

How Does Gradient Descent Work?

- 1.Gradient descent is an optimization algorithm used to minimize the cost function of a model.
- 2.The cost function measures how well the model fits the training data and is defined based on the difference between the predicted and actual values.
- 3.The gradient of the cost function is the derivative with respect to the model's parameters and points in the direction of the steepest ascent.
- 4.The algorithm starts with an initial set of parameters and updates them in small steps to minimize the cost function.
- 5.In each iteration of the algorithm, the gradient of the cost function with respect to each parameter is computed.

6. The gradient tells us the direction of the steepest ascent, and by moving in the opposite direction, we can find the direction of the steepest descent.
7. The size of the step is controlled by the learning rate, which determines how quickly the algorithm moves towards the minimum.
8. The process is repeated until the cost function converges to a minimum, indicating that the model has reached the optimal set of parameters.
9. There are different variations of gradient descent, including batch gradient descent, stochastic gradient descent, and mini-batch gradient descent, each with its own advantages and limitations.
10. Efficient implementation of gradient descent is essential for achieving good performance in machine learning tasks. The choice of the learning rate and the number of iterations can significantly impact the performance of the algorithm.

Gradient Descent

- **Advantages:**

1. Easy computation.
2. Easy to implement.
3. Easy to understand.

- **Disadvantages:**

1. May trap at local minima.
2. Weights are changed after calculating the gradient on the whole dataset. So, if the dataset is too large then this may take years to converge to the minima.
3. Requires large memory to calculate the gradient on the whole dataset.

Stochastic Gradient Descent (SGD)

- SGD algorithm is an extension of the Gradient Descent and it overcomes some of the disadvantages of the GD algorithm.
- Gradient Descent has a disadvantage that it requires a lot of memory to load the entire dataset of n -points at a time to compute the derivative of the loss function.
- **In the SGD algorithm derivative is computed taking one point at a time.**

Stochastic Gradient Descent (SGD)

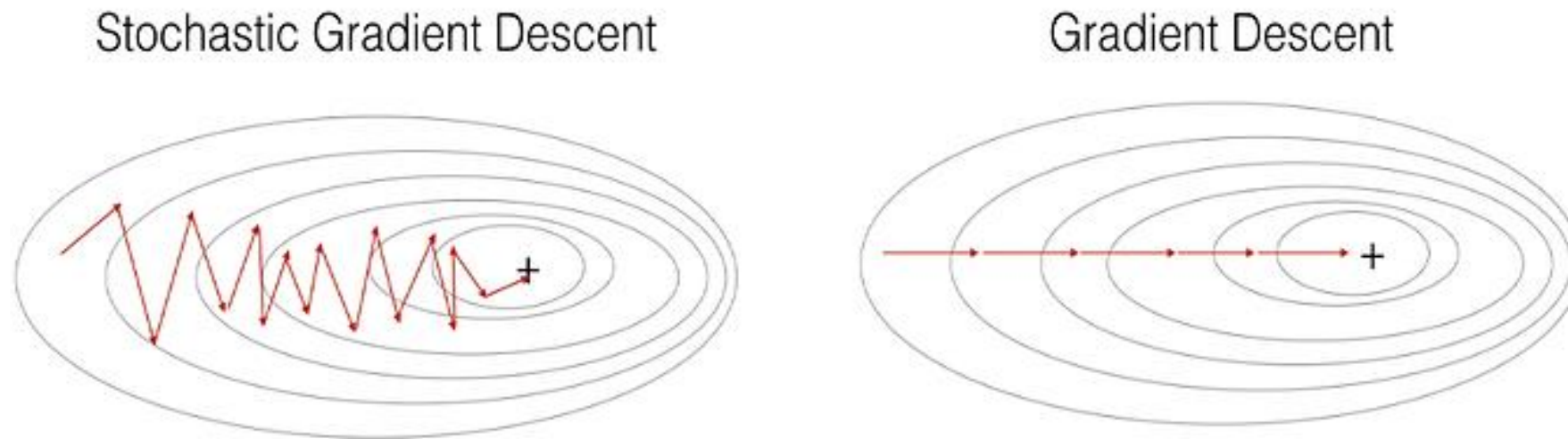


Figure 1 : SGD vs GD

"+" denotes a minimum of the cost. SGD leads to many oscillations to reach convergence. But each step is a lot faster to compute for SGD than for GD, as it uses only one training example (vs. the whole batch for GD).

- **Advantage:**

- Memory requirement is less compared to the GD algorithm as the derivative is computed taking only 1 point at once.

- **Disadvantages:**

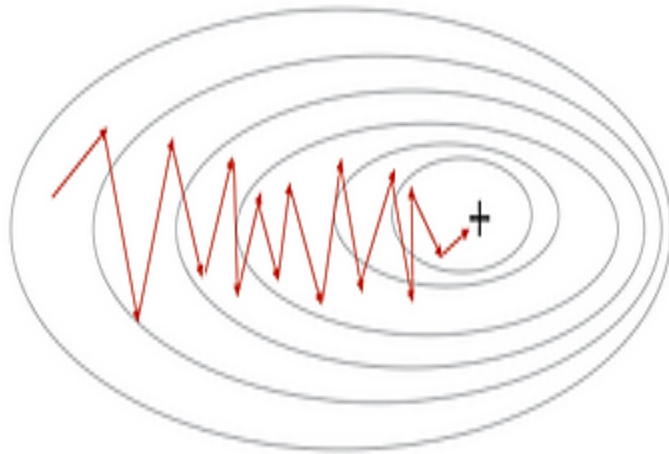
1. The time required to complete 1 epoch is large compared to the GD algorithm.
2. Takes a long time to converge.
3. May stuck at local minima.

Mini Batch Stochastic Gradient Descent (MB-SGD)

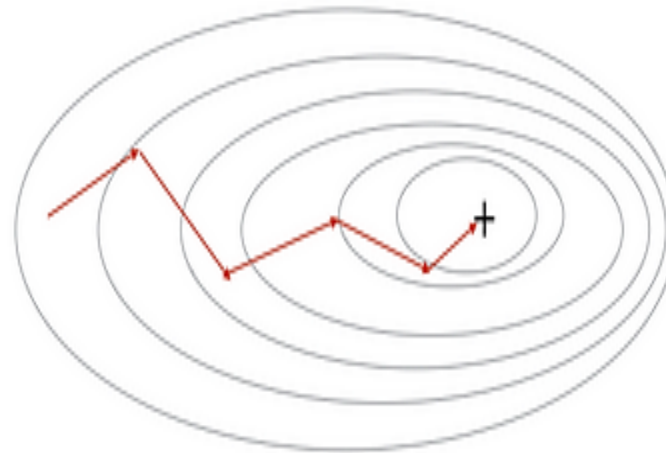
- MB-SGD algorithm is an extension of the SGD algorithm and it overcomes the problem of large time complexity in the case of the SGD algorithm.
- MB-SGD algorithm takes a batch of points or subset of points from the dataset to compute derivative.
- It is observed that the derivative of the loss function for MB-SGD is almost the same as a derivative of the loss function for GD after some number of iterations.
- But the number of iterations to achieve minima is large for MB-SGD compared to GD and the cost of computation is also large.

Mini Batch Stochastic Gradient Descent (MB-SGD)

Stochastic Gradient Descent



Mini-Batch Gradient Descent



- **Advantages:**

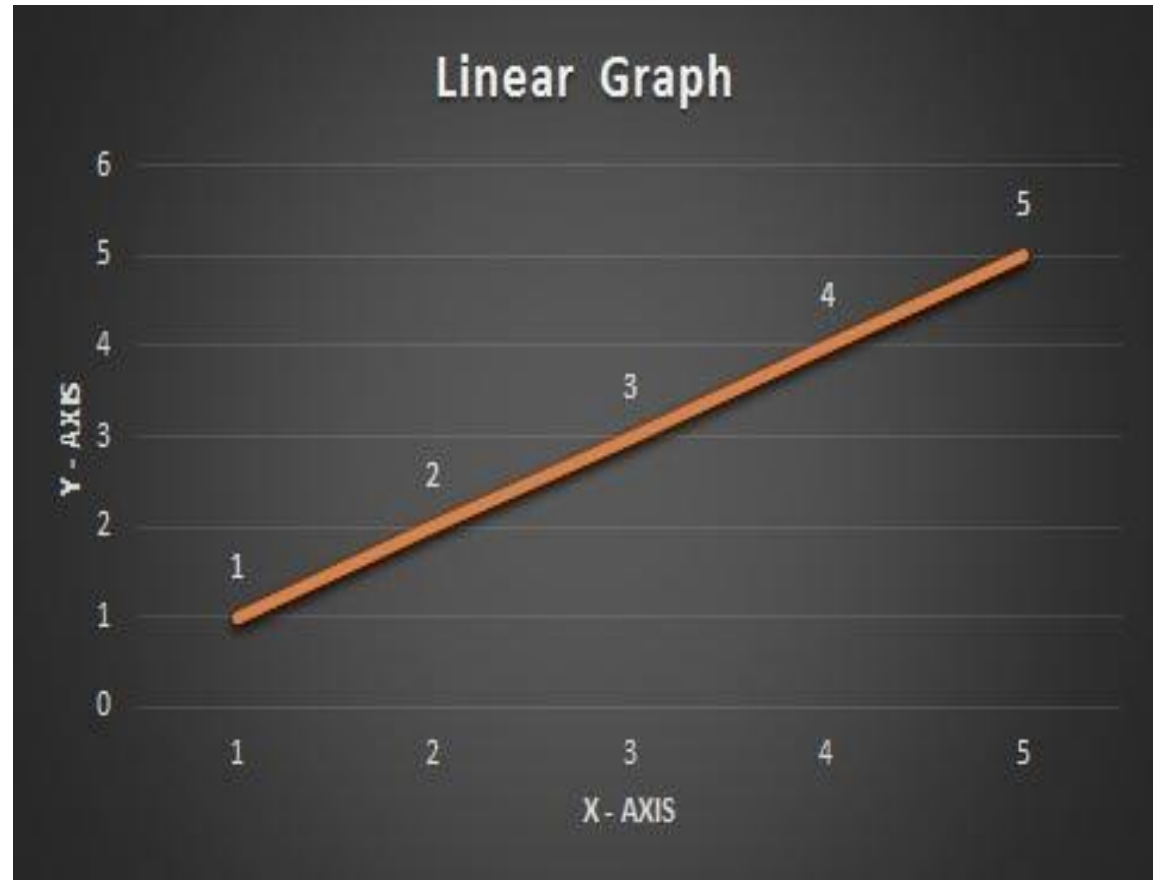
- Less time complexity to converge compared to standard SGD algorithm.

- **Disadvantages:**

1. The update of MB-SGD is much noisy compared to the update of the GD algorithm.
2. Take a longer time to converge than the GD algorithm.
3. May get stuck at local minima.

- What is Linear Regression?
- Linear Regression is a predictive algorithm which provides a Linear relationship between Prediction (Call it 'Y') and Input (Call is 'X').

- (Input) $X = 1, 2, 3, 4, 5$
(Prediction) $Y = 1, 2, 3, 4, 5$



Agricultural Product

Price @ Point Of Sale



Farmer (1)	→	4
Village(2)	→	12
Town(3)	→	28
City(4)	→	52
City Downtown(5)	→	80

Training Data Set



- **Training DataSet**

- The dataset provided above can be considered as Training DataSet for the problem statement stated above, If we consider these inputs as Training Data for the model, we can use that model to predict the price at locations between
 - Farmers home — Village
 - Village — Town
 - Town — City
 - City — City Downtown

- Equation of Straight Line ($y = mx + b$)
- The maths allow us to get a straight line between any two (x,y) points in two dimensional graph. For this example, let's consider farmers home and price as starting point and city downtown as ending point.
- The coordinates of the start and end points will be
- $(x_1, y_1) = (1, 4)$
- $(x_2, y_2) = (5, 80)$
- where x represents the location and y represent the price.

Agricultural Product	Price @ Point Of Sale
Farmer (1)	4
Village(2)	12
Town(3)	28
City(4)	52
City Downtown(5)	80

- The first step is to come up with a formula in the form of $y = mx + b$ where x is a known value and y is the predicted value.
- To calculate the Prediction y for any Input value x we have two unknowns, the $m = \text{slope(Gradient)}$ and $b = \text{y-intercept(also called bias)}$

- Slope ($m = \text{Change in } y / \text{Change in } x$)
- The slope of the line is calculated as the change in y divided by change in x , so the calculation will look like

Given $(x_1, y_1) = (1, 4)$ and $(x_2, y_2) = (5, 80)$

$m = \text{Change in } Y / \text{Change in } X$

$$m = (y_2 - y_1) / (x_2 - x_1)$$

$$m = (80 - 4) / (5 - 1)$$

$$m = 76 / 4$$

$$m = 19$$

Given $m = 19$ and $(x_1, y_1) = (1, 4)$

$$y - y_1 = m(x - x_1)$$

$$y - 4 = 19(x - 1)$$

$$y - 4 = 19x - 19$$

$$y = 19x - 19 + 4$$

$$y = 19x - 15$$

This can be written in the form of $y = mx + b$ as

$$y = 19x + (-15), \text{ so } b = -15$$

Given Formula $\{ y = 19x + (-15) \}$

$$x_1 \Rightarrow 1 \qquad \{ y = 19 * 1 - 15 \Rightarrow 19 - 15 \Rightarrow 4 \text{ (i.e. } y_1 \text{)} \}$$

$$x_2 \Rightarrow 5 \qquad \{ y = 19 * 5 - 15 \Rightarrow 95 - 15 \Rightarrow 80 \text{ (i.e. } y_2 \text{)} \}$$

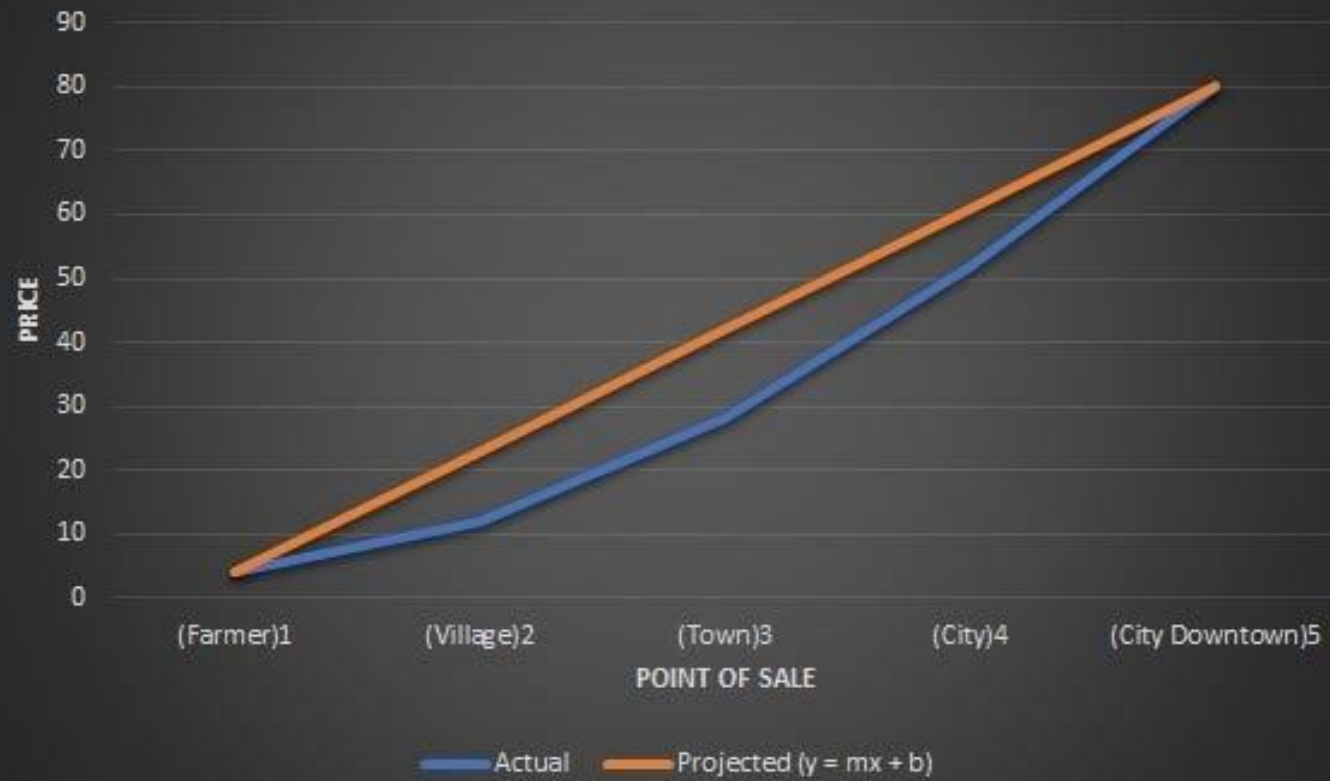
Given Formula $\{y = 19x + (-15)\}$

$$x \Rightarrow 2 \quad \{y = 19 * 2 - 15 \Rightarrow 38 - 15 \Rightarrow 23\}$$

$$x \Rightarrow 3 \quad \{y = 19 * 3 - 15 \Rightarrow 57 - 15 \Rightarrow 42\}$$

$$x \Rightarrow 4 \quad \{y = 19 * 4 - 15 \Rightarrow 76 - 15 \Rightarrow 61\}$$

Actual Vs Straight Line



Least Square Regression

$$m = \frac{\text{Sum of ALL } (x - x_{\text{mean}}) * (y - y_{\text{Mean}})}{\text{Sum of } (x - x_{\text{mean}})^2}$$

To Calculate $\Rightarrow (x - x_{\text{mean}})$

Mean Value of all 'x' $\Rightarrow (1 + 2 + 3 + 4 + 5) / 5 = 3$

At $x = 1$: $(x - x_{\text{mean}}) \Rightarrow (1 - 3) = -2$

At $x = 2$: $(x - x_{\text{mean}}) \Rightarrow (2 - 3) = -1$

At $x = 3$: $(x - x_{\text{mean}}) \Rightarrow (3 - 3) = 0$

At $x = 4$: $(x - x_{\text{mean}}) \Rightarrow (4 - 3) = 1$

At $x = 5$: $(x - x_{\text{mean}}) \Rightarrow (5 - 3) = 2$

To Calculate $\Rightarrow (y - y_{\text{mean}})$

Mean Values of all 'y' $\Rightarrow (4 + 12 + 28 + 52 + 80) / 5 = 35.2$

At $y = 1$: $(y - y_{\text{mean}}) \Rightarrow (4 - 35.2) = -31.2$

At $y = 2$: $(y - y_{\text{mean}}) \Rightarrow (12 - 35.2) = -23.2$

At $y = 3$: $(y - y_{\text{mean}}) \Rightarrow (28 - 35.2) = -7.2$

At $y = 4$: $(y - y_{\text{mean}}) \Rightarrow (52 - 35.2) = 16.8$

At $y = 5$: $(y - y_{\text{mean}}) \Rightarrow (80 - 35.2) = 44.8$

To Calculate => SUM OF ALL $\{ (x - \bar{x}) * (y - \bar{y}) \}$

$$(x,y) \Rightarrow (1,4) : (x - \bar{x}) * (y - \bar{y}) \Rightarrow \{ -2 * -31.2 = 62.4 \}$$

$$(x,y) \Rightarrow (2,12) : (x - \bar{x}) * (y - \bar{y}) \Rightarrow \{ -1 * -23.2 = 23.2 \}$$

$$(x,y) \Rightarrow (3,28) : (x - \bar{x}) * (y - \bar{y}) \Rightarrow \{ 0 * -7.2 = 0 \}$$

$$(x,y) \Rightarrow (4,52) : (x - \bar{x}) * (y - \bar{y}) \Rightarrow \{ 1 * 16.8 = 16.8 \}$$

$$(x,y) \Rightarrow (4,52) : (x - \bar{x}) * (y - \bar{y}) \Rightarrow \{ 2 * 44.8 = 89.6 \}$$

$$\text{Sum of All} \Rightarrow \{ 62.4 + 23.2 + 0 + 16.8 + 89.6 \} = 192$$

To Calculate $\Rightarrow$ Sum of $(x - \bar{x})^2$

$$\{-2^2, -1^2, 0^2, 1^2, 2^2\} =$$

$$\{4, 1, 0, 1, 4\} =$$

$$\{10\}$$

$$m = \frac{\text{Sum of ALL } (x - x_{\text{mean}}) * (y - y_{\text{mean}})}{\text{Sum of } (x - x_{\text{mean}})^2}$$

$$m = 192 / 10 = 19.2$$

To Calculate $\Rightarrow b = y_{\text{mean}} - m * x_{\text{mean}}$

$$b = 35.2 - 19.2 * 3$$

$$b = 35.2 - 57.6$$

$$b = -22.4$$

$$y = mx + b \Rightarrow \{ y = 19.2x + (-22.4) \}$$

Given Formula $\Rightarrow \{ y = 19.2x + (-22.4) \}$

$$x \Rightarrow 1 \quad \{ y = 19.2 * 1 - 22.4 \} = -3.2$$

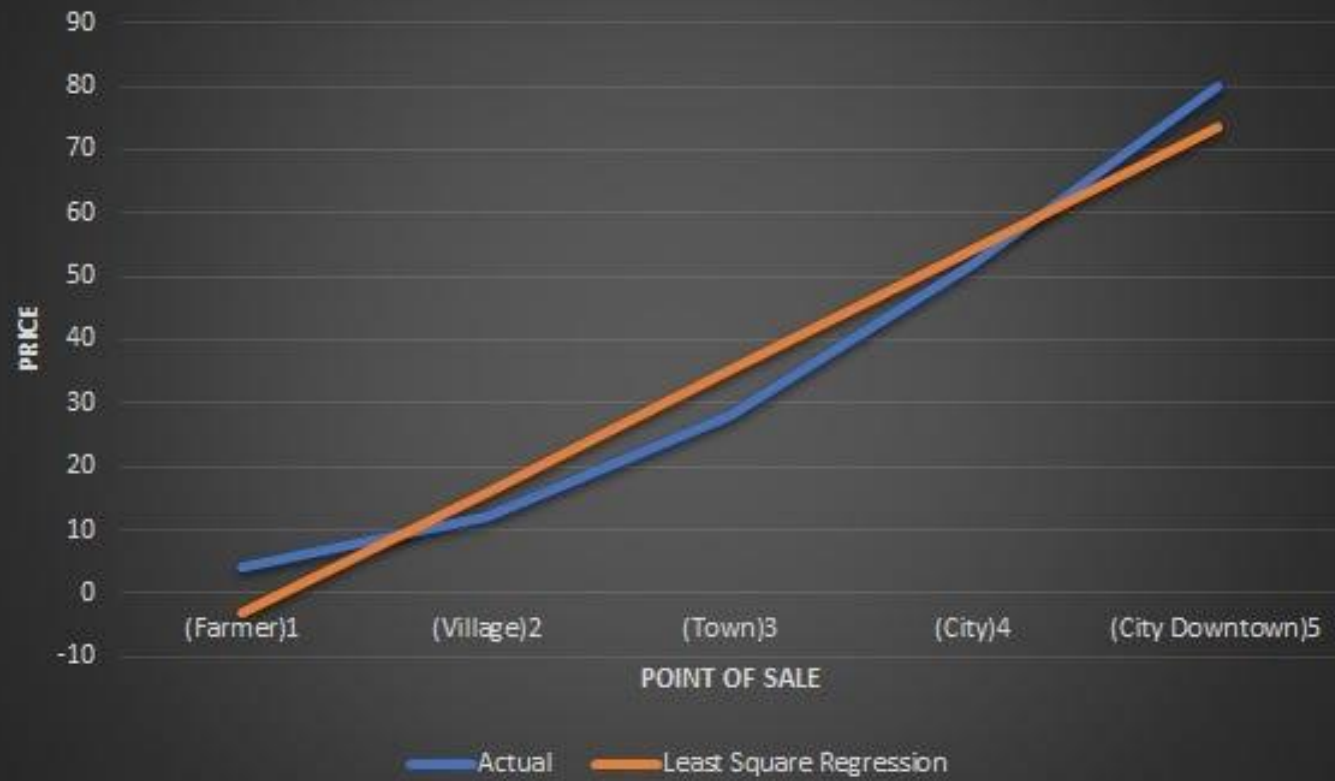
$$x \Rightarrow 2 \quad \{ y = 19.2 * 2 - 22.4 \} = 16$$

$$x \Rightarrow 3 \quad \{ y = 19.2 * 3 - 22.4 \} = 35.2$$

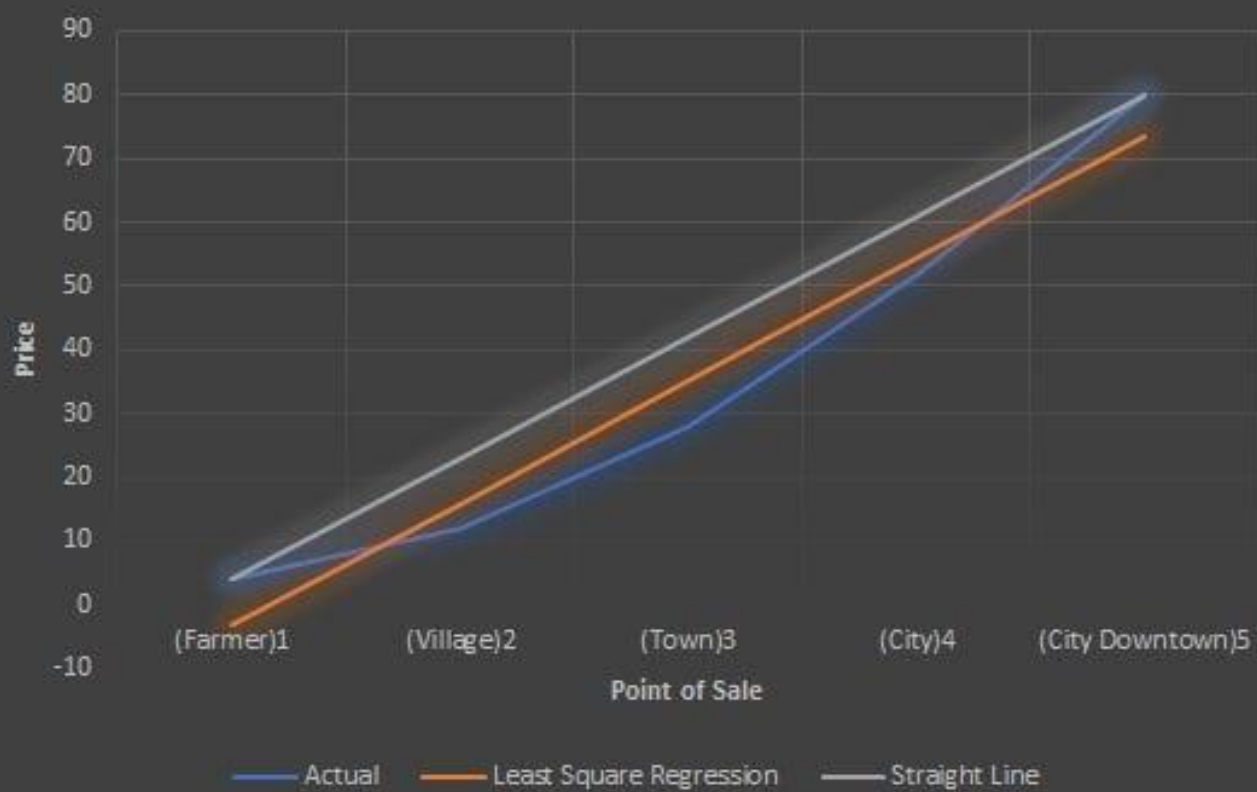
$$x \Rightarrow 4 \quad \{ y = 19.2 * 4 - 22.4 \} = 54.4$$

$$x \Rightarrow 5 \quad \{ y = 19.2 * 5 - 22.4 \} = 73.6$$

Actual Vs Least Square



Actual Vs Least Square Vs Straight Line



Error in 'Y' With Straight Line Equation

At X-Value	Y-Value	Actual-Value	Error	Square-Error
1	4	4	0	0
2	12	23	-11	121
3	28	42	-14	196
4	52	61	-9	81
5	80	80	0	0

$$\text{Sum of Square Error} = \{ 0 + 121 + 196 + 81 + 0 \} = 398$$

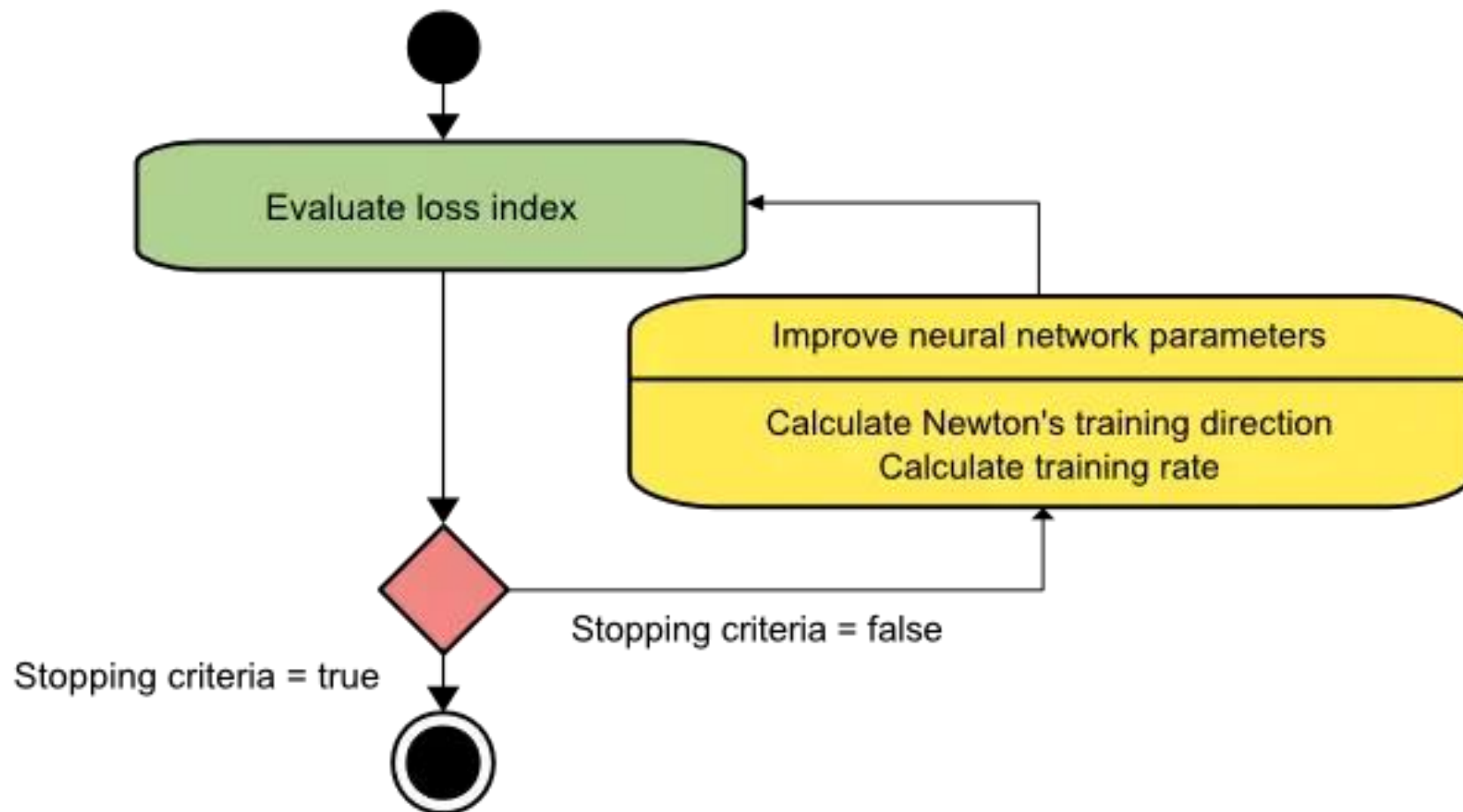
Error in 'Y' With Least Square Equation

At X-Value	Y-Value	Actual-Value	Error	Square-Error
1	4	-3.2	7.2	51.84
2	12	16	-4	16
3	28	35.2	-7.2	51.84
4	52	54.4	-2.4	5.76
5	80	73.6	6.4	40.96

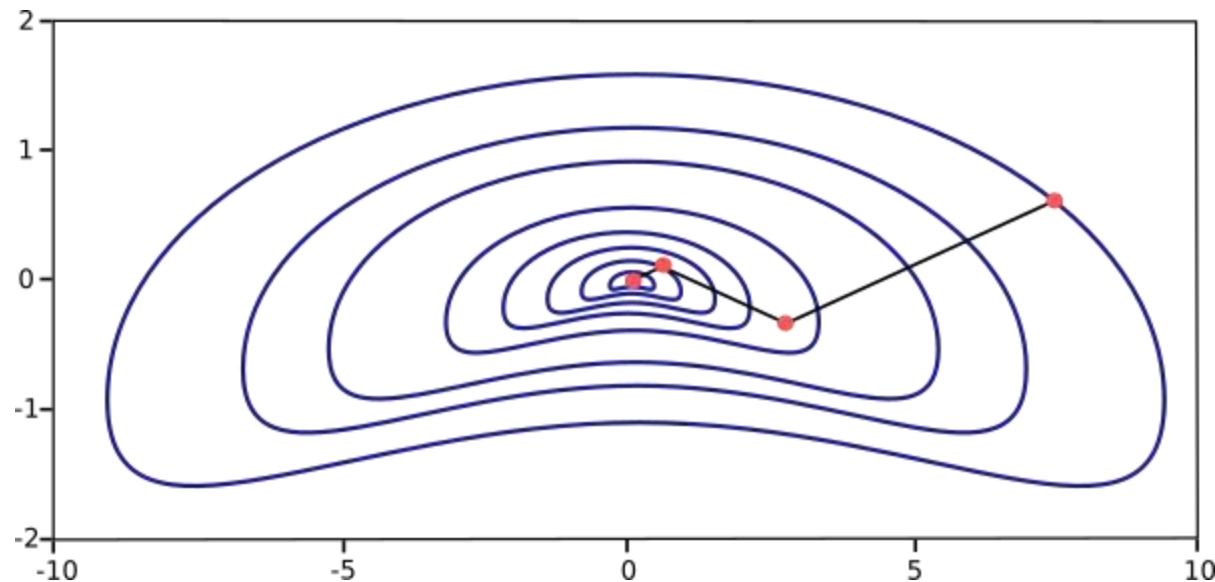
Sum of Square Error = { 51.84 + 16 + 51.84 + 5.76 + 40.96 } = 166.4

Newton's method (NM)

- Newton's method is a second-order algorithm because it uses the Hessian matrix.
- This method's objective is to find better training directions by using the second derivatives of the [loss](#) function.

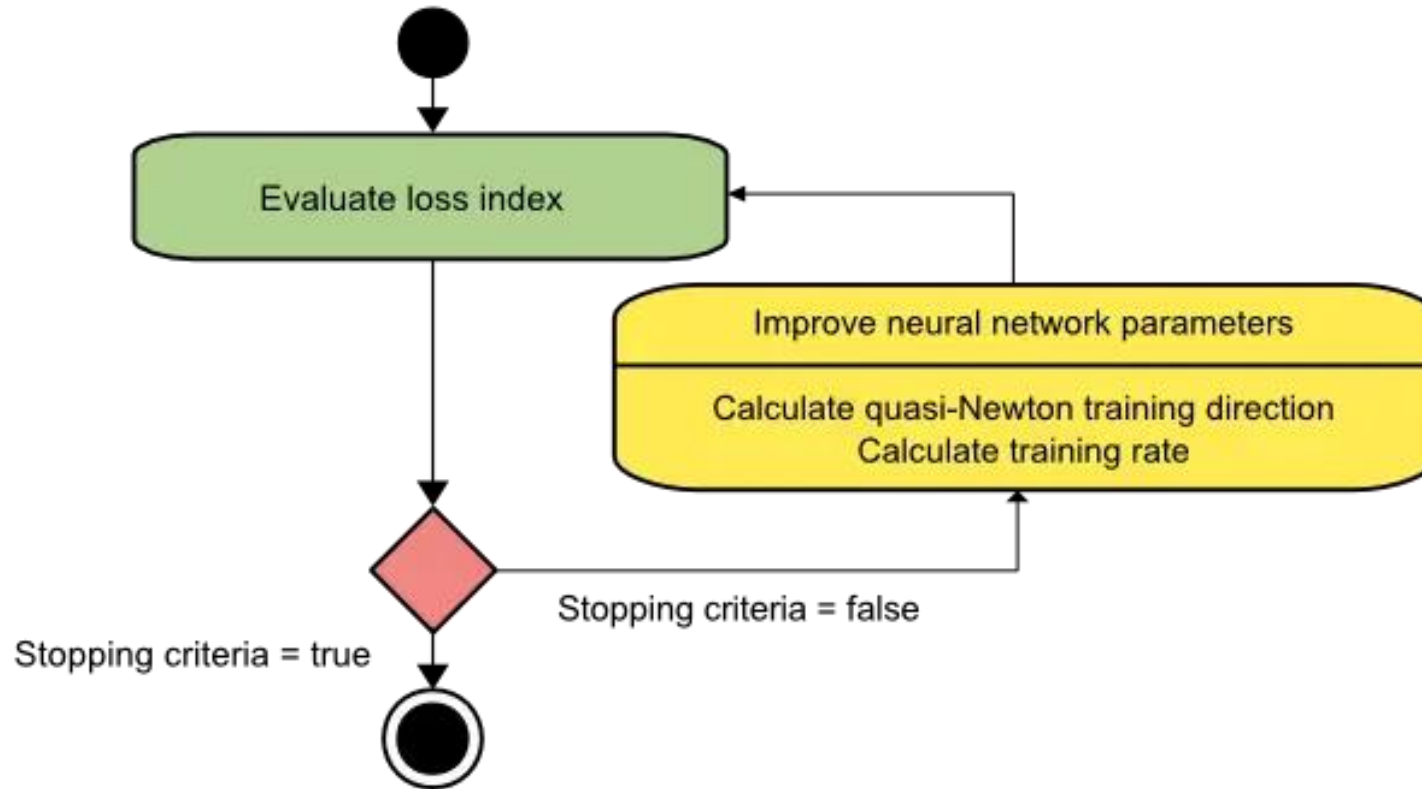


- The picture below illustrates the performance of this method.
- As we can see, Newton's method requires fewer steps than gradient descent to find the minimum value of the [loss](#) function.
- Newton's method has the difficulty that the exact evaluation of the Hessian and its inverse are pretty expensive in computational terms.



Quasi-Newton method (QNM)

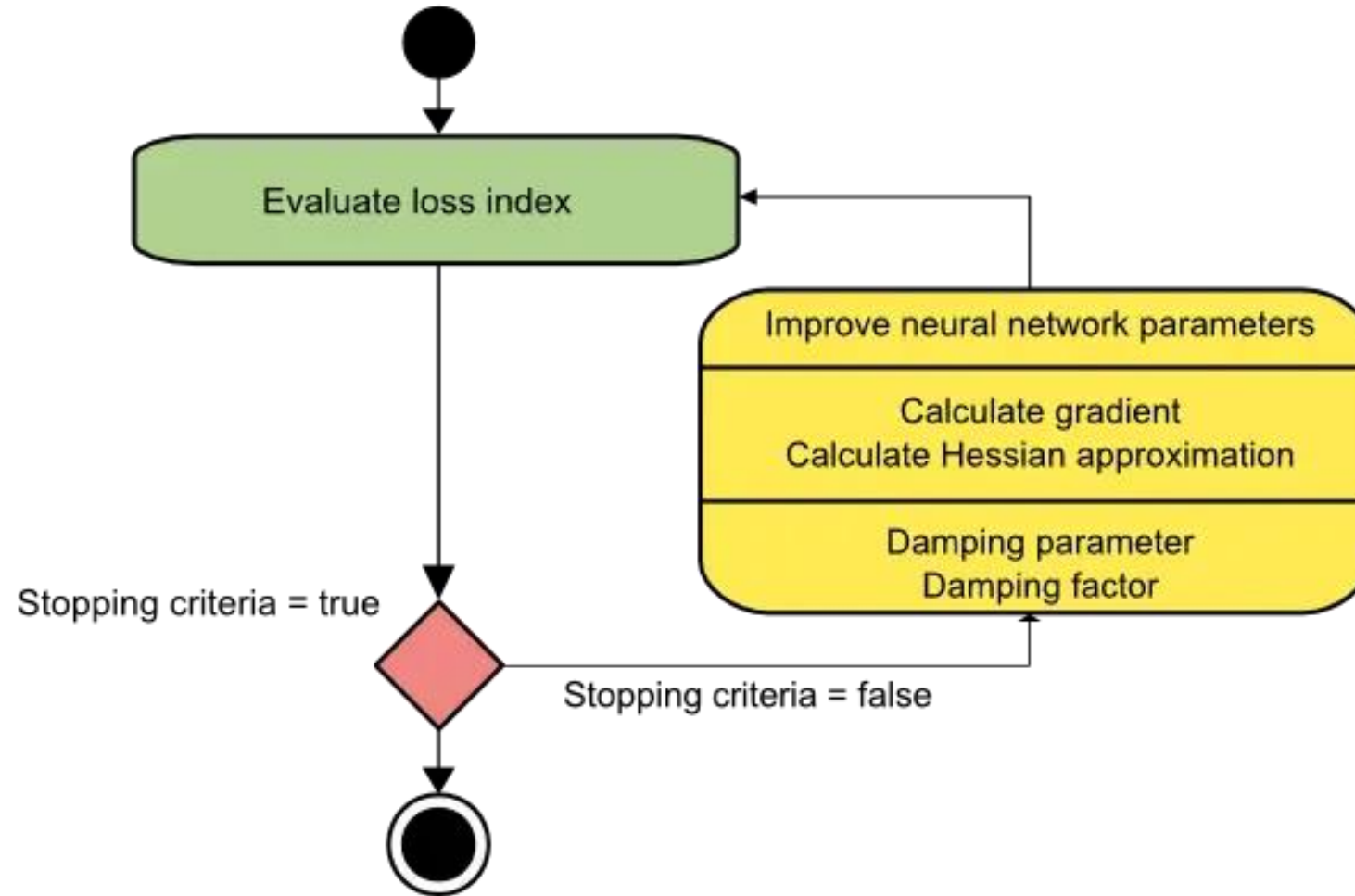
- The application of Newton's method is computationally expensive.
- Indeed, it requires many operations to evaluate the Hessian matrix and compute its inverse.
- Alternative approaches, known as [quasi-Newton](#), are developed to solve that drawback.
- These methods do not calculate the Hessian directly and then evaluate its inverse.
- Instead, they build up an approximation to the inverse Hessian.
- This approximation is computed using only information on the first derivatives of the [loss function](#).



This is the default method to use in most cases: It is faster than [gradient descent](#) and [conjugate gradient](#), and the exact Hessian does not need to be computed and inverted.

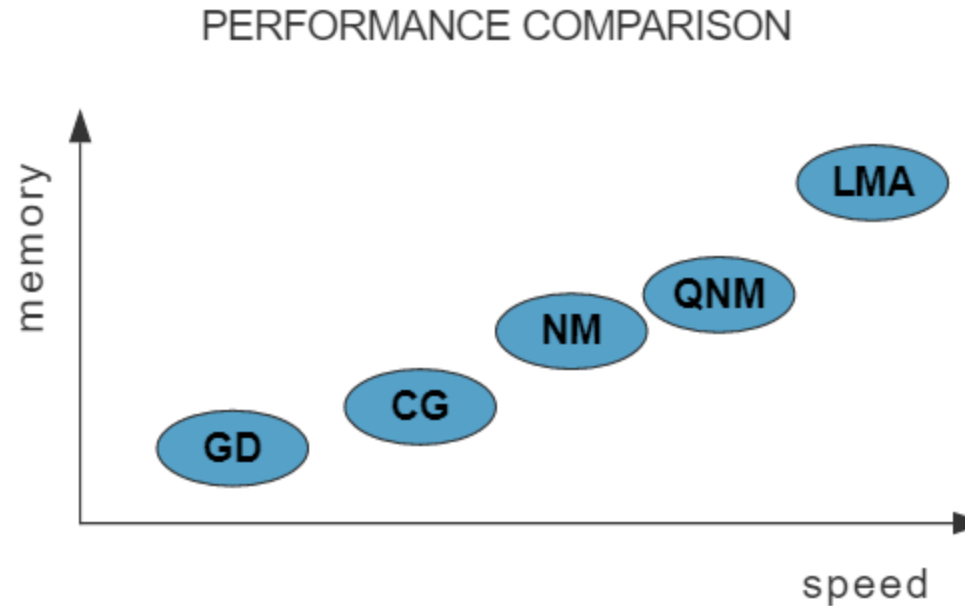
Levenberg-Marquardt algorithm (LM)

- The [Levenberg-Marquardt algorithm](#) is designed to work specifically with loss functions which take the form of a sum of squared errors.
- It works without computing the exact Hessian matrix.
- Instead, it works with the gradient vector and the Jacobian matrix.



- The [Levenberg-Marquardt algorithm](#) is a method tailored for functions of the type sum-of-squared-error.
- That makes it to be very fast when training [neural networks](#) measured on that kind of errors.
- However, this algorithm has some drawbacks.
- First, it cannot minimize functions such as the root mean squared error or the cross-entropy error.
- Also, the Jacobian matrix becomes enormous for big data sets and neural networks, requiring much memory.
- Therefore, the [Levenberg-Marquardt algorithm](#) is not recommended when we have big data sets or neural networks.

Performance comparison



- The slowest training algorithm is usually gradient descent, but it is the one requiring less memory.
- On the contrary, the fastest one might be the Levenberg-Marquardt algorithm, but it usually requires much memory.
- A good compromise might be the quasi-Newton method.

1. It is observed that the SGD algorithm (red) is stuck at a saddle point. So SGD algorithm can only be used for shallow networks.
2. All the other algorithms except SGD finally converge one after the other, AdaDelta being the fastest followed by momentum algorithms.
3. AdaGrad and AdaDelta algorithm can be used for sparse data.
4. Momentum and NAG work well for most cases but is slower.
5. Animation for Adam is not available but from the plot above it is observed that it is the fastest algorithm to converge to minima.
6. Adam is considered the best algorithm amongst all the algorithms discussed above.